핸즈온 사전 준비 가이드

Build Your Own AI Office with Gemma 4

세션명	Build Your Own AI Office with Gemma 4
진행자	박제창
행사	Build with AI Seoul 2026
트랙	Hands-On A
진행 시간	13:30 ~ 14:30 (60분)
수록 범위	docs/01-15 전체 문서 원문
기준 확인일	2026-04-21

이 PDF는 저장소의 01-15번 사전 준비 문서를 순서대로 하나의 배포본으로 묶은 버전입니다.

표지 다음 페이지부터 각 문서의 본문을 전체 수록합니다.

한장 요약: 행사 전 준비

행사 당일에는 설치와 모델 다운로드를 시작하지 않는 것을 기준으로 준비합니다. 아래 항목 중 본인 환경에 맞는 경로를 선택하고, 모델 다운로드와 1회 실행 테스트까지 끝내 주세요.

가장 중요한 기준

하나만 설치한다면 LM Studio를 우선 권장합니다. ChromeOS 또는 Intel Mac은 Ollama를 우선 고려하세요.

8GB 장비는 E2B only, best-effort입니다. 가능하면 16GB 이상 장비로 참석해 주세요.

설치해야 하는 소프트웨어와 LLM

참가자 유형	설치 소프트웨어	미리 받을 LLM	완료 기준
대부분의 참가자	LM Studio	Gemma 4 E4B 권장, 8GB는 E2B	앱 실행, 모델 다운로드, Chat 1회 성공
CLI/API도 필요한 참가자	LM Studio + Ollama	`gemma4:e2b` 또는 `gemma4:e4b`	`ollama run` 1회 성공
ChromeOS / Intel Mac	Ollama	`gemma4:e2b` 우선	Linux 환경 또는 터미널에서 1회 실행
고급 에이전트 실습	LM Studio 또는 llama.cpp + OpenCode/Hermes	32GB+는 26B A4B, 36GB+는 31B 가능	local endpoint와 에이전트 연결 확인

메모리별 LLM 선택

노트북 메모리	권장 LLM	권장 도구	준비 메모
8GB	Gemma 4 E2B만	LM Studio 또는 Ollama	속도 저하 가능, 다른 앱 종료
16GB	Gemma 4 E4B	LM Studio 우선	가장 무난한 참가 사양
32GB	E4B 또는 26B A4B	LM Studio, 필요 시 llama.cpp	에이전트 실습은 26B A4B 권장
36GB+	26B A4B 또는 31B	LM Studio 또는 llama.cpp	품질 우선은 31B, 반응성 우선은 26B A4B

공통 최종 체크리스트

- 설치 권한 또는 관리자 권한 확인
- 디스크 여유 공간 최소 20GB, 가능하면 40GB 이상 확보
- 선택한 소프트웨어 설치 완료
- 사용할 Gemma 4 모델 다운로드 완료
- 행사 전 최소 1회 응답 생성 테스트 완료
- 실습 당일 전원 어댑터 지참

수록 문서

아래 문서를 순서대로 모두 포함했습니다. 각 문서는 다음 페이지부터 원문 내용을 전체 수록합니다.

- 01. 하드웨어 / 운영체제 / 모델 선택 가이드
- 02. Windows 참가자 빠른 준비 가이드
- 03. 메모리 기준으로 Gemma 4 모델 고르는 방법
- 04. GGUF, MLX, llama.cpp 개념 설명
- 05. LM Studio 설치 가이드
- 06. Ollama 설치 가이드
- 07. llama.cpp 설치 가이드
- 08. Apple Silicon + MLX 안내
- 09. Gemma 4 벤치마크와 코딩 에이전트 기대치
- 10. OpenCode 설치 및 LM Studio 연동 가이드
- 11. Pi 경량 코딩 에이전트 대안과 이번 핸즈온 선택 이유
- 12. Hermes Agent란 무엇인가
- 13. Hermes Agent 설치 가이드
- 14. Gemma 4 아키텍처 상세 정리
- 15. 트러블슈팅 / 최종 체크 / 참고 링크

편집 방침

세션명과 진행자명은 표지에 반영했습니다.

기존 요약 위주의 본문은 제거하고, 각 마크다운 문서의 전체 내용을 넣도록 생성 방식을 바꿨습니다.

문서 내부의 상대 링크는 PDF 안에서는 링크 라벨 텍스트로 정리해 표시합니다.

문서 01

docs/01-hardware-and-model-selection.md

하드웨어 / 운영체제 / 모델 선택 가이드

이 문서에서 다루는 내용

- 어떤 도구를 기본으로 설치할지
- 운영체제별 권장 경로
- 8GB, 16GB, 32GB, 36GB 메모리별 권장 모델
- 행사 전에 어떤 모델을 미리 받아야 하는지

추가 참고:

- 코딩 에이전트 기대치와 모델별 벤치마크는 Gemma 4 벤치마크와 코딩 에이전트 기대치 문서를 확인해 주세요.
- 메모리로 어떤 모델을 골라야 하는지는 메모리 기준으로 Gemma 4 모델 고르는 방법 문서를 확인해 주세요.

- OpenCode를 개발자 에이전트처럼 연결해 보고 싶다면 OpenCode 설치 및 LM Studio 연동 가이드 문서를 확인해 주세요.
- Hermes Agent가 정확히 어떤 도구인지 먼저 이해하고 싶다면 Hermes Agent란 무엇인가 문서를 확인해 주세요.

Ollama와 LM Studio 비교

항목	Ollama	LM Studio
추천 대상	개발자, CLI/API 선호 사용자	개발자, 비개발자 모두 / GUI 선호 사용자
설치 방식	앱 + CLI 중심	데스크톱 앱 중심
모델 받기	`ollama pull` 또는 `ollama run`	Discover 탭에서 검색/다운로드
사용 방식	터미널, 로컬 API, 자동화에 강함	채팅 UI, 모델 관리 UI, 로컬 서버에 강함
오프라인 사용	모델 다운로드 후 로컬 실행	모델 다운로드 후 완전 오프라인 사용 가능
macOS 지원	Apple Silicon 지원, Intel Mac은 CPU 전용	Apple Silicon만 지원, Intel Mac 미지원
Windows 지원	네이티브 앱, NVIDIA/AMD GPU 지원	x64/ARM 지원, 16GB+ RAM 권장
ChromeOS	Linux 개발 환경에서 가능하지만 느릴 수 있음	공식 권장 경로 아님
행사 권장도	보조 또는 고급/CLI 경로	기본 권장

운영체제별 빠른 추천

운영체제	가장 권장	대안	비고
Windows	LM Studio	Ollama	하나만 설치한다면 LM Studio를 권장합니다.
macOS Apple Silicon	LM Studio	Ollama	LM Studio는 Apple Silicon 전용이며, GGUF를 기본으로 쓰고 필요하면 MLX도 활용할 수 있습니다.
macOS Intel	Ollama	없음에 가까움	LM Studio 데스크톱 앱은 Intel Mac 미지원입니다.
Linux (Ubuntu)	LM Studio	Ollama	GUI 설치가 가능하면 LM Studio를 우선 권장합니다.
ChromeOS	Ollama	가능하면 다른 노트북 준비	Linux 개발 환경에서는 로컬 LLM용 전용 GPU/VRAM 가속을 기대하기 어렵습니다.

메모리별 권장 모델

아래 권장은 행사 당일 안정적인 진행을 기준으로 잡았습니다. 모델 페이지의 표시 크기나 최소 메모리는 더 낮게 보일 수 있지만, 실제로는 운영체제와 브라우저도 메모리를 함께 사용하므로 여유가 필요합니다.

노트북 메모리	행사 권장 모델	권장 도구	안내
8GB	`gemma4:e2b` / `google/gemma-4-e2b`	E2B만 시도	8GB는 공식 권장 사양보다 낮아 LM Studio/Ollama 어느 쪽도 무난한 기본 조합으로 보기 어렵습니다. E2B만 시도하고, 가능하면 16GB 이상 노트북을 권장합니다. 실제로는 속도가 많이 느리거나 시스템이 일시적으로 멈춘 것처럼 보일 수 있습니다.
16GB	`gemma4:e4b` / `google/gemma-4-e4b`	LM Studio 우선	가장 무난한 참가 사양입니다. 실패 시 E2B로 낮추면 됩니다.
32GB	`gemma4:e4b`, `gemma4:26b` / `google/gemma-4-26b-a4b`	LM Studio 우선	속도와 안정성을 우선하면 E4B, 더 큰 모델을 원하면 26B A4B를 선택하세요.
36GB	`gemma4:26b`, `gemma4:31b` / `google/gemma-4-31b`	LM Studio 우선	31B도 가능하지만, 반응 속도는 26B A4B가 더 나을 수 있습니다.

A4B는 무슨 뜻인가요?

26B A4B에서 26B는 모델 전체 파라미터 규모를 뜻하고, A4B는 답을 만들 때 실제로 계산에 참여하는 부분이 약 4B 규모라는 뜻입니다.

이 방식은 `MoE(Mixture-of-Experts)` 구조라고 부릅니다. 모델 안에 여러 "전문가" 부분이 있고, 답을 만들 때마다 필요한 일부 전문가만 골라 계산하는 방식입니다. 공식 모델 카드 기준으로는 총 파라미터가 `25.2B`, 실제 계산에 참여하는 파라미터가 `3.8B`입니다.

그래서 26B A4B는 31B처럼 모든 주요 계산 블록을 함께 쓰는 모델보다 더 빠르게 느껴질 수 있습니다. 다만 모델 파일은 전체 26B 규모를 메모리에 올려야 하므로, 4B 모델처럼 가볍게 실행된다는 뜻은 아닙니다. 32GB 이상 장비에서 권장하는 이유가 이 점 때문입니다.

8GB 추가 메모:

- Windows와 macOS에서 LM Studio 공식 권장 메모리는 16GB 이상입니다.
- 따라서 8GB는 "무난한 기본 조합"이 아니라, 작은 모델(E2B)만 시도하는 최소 준비로 이해해 주세요.
- 8GB Apple Silicon Mac도 E2B/E4B 실행 중 속도가 크게 떨어지거나 시스템이 멈춘 것처럼 보일 수 있으므로, 안정적인 실습 장비로 보기 어렵습니다.
- 8GB 장비로 참석하는 경우에는 작은 컨텍스트, 브라우저 탭 최소화, 다른 앱 종료를 전제로 준비해 주세요.

로컬 모델 크기 참고

LM Studio Gemma 4 모델 페이지 기준 표시 크기:

- `google/gemma-4-e2b`: 4.20GB
- `google/gemma-4-e4b`: 5.90GB
- `google/gemma-4-26b-a4b`: 17.00GB
- `google/gemma-4-31b`: 19.00GB

Ollama Gemma 4 태그 페이지 기준으로는 기본 태그가 `gemma4:e2b` 7.2GB, `gemma4:e4b` 9.6GB, `gemma4:26b` 18GB, `gemma4:31b` 20GB로 표시됩니다.

이번 핸즈온 권장은 위 표시 크기보다 여유 있게 잡았습니다. 모델 크기 외에도 실행 도구, 긴 대화 컨텍스트, 브라우저, 운영체제가 메모리를 함께 사용합니다.

어떤 모델을 미리 받아두면 좋을까?

행사 표준 권장안은 아래와 같습니다.

중요:

- 이번 핸즈온은 채팅 / 질의응답 / 에이전트 흐름이 중심이므로, 가능하면 instruction-tuned (`-it`) 또는 chat-ready 모델을 받으세요.
- Hugging Face나 `llama.cpp` 예시에서는 보통 `gemma-4-...-it` 처럼 보입니다.
- Ollama는 `gemma4:e2b`, `gemma4:e4b`, `gemma4:26b`, `gemma4:31b` 같은 런타임 태그를 쓰므로, 사용자가 별도로 `-it` 접미사를 고를 필요는 없습니다.
- LM Studio는 모델 목록 이름이 `google/gemma-4-e4b`처럼 더 짧게 보일 수 있으므로, 채팅/실습용 instruction-tuned 또는 chat-ready 항목인지 설명을 함께 확인해 주세요.

- 8GB: E2B만 미리 다운로드
- 8GB는 현장 안정성이 낮으므로, 가능하면 16GB 이상 장비로 변경
- 16GB: E4B 다운로드, 불안하면 E2B도 함께 준비
- 32GB: E4B 또는 26B A4B
- 36GB: 26B A4B 또는 31B

중요:

- 행사 당일에 모델 다운로드를 시작하지 마세요.
- 모델 파일이 커서 네트워크 상황이 좋지 않으면 실습 시간 대부분을 다운로드에 쓰게 될 수 있습니다.
- 최소한 설치, 모델 다운로드, 1회 실행 테스트까지 끝낸 상태로 참석해 주세요.

권장 준비 시나리오

가장 안전한 시나리오

- Windows/macOS/Linux 사용자: LM Studio 설치 + chat-ready / instruction-tuned `google/gemma-4-e2b` 또는 `google/gemma-4-e4b` 다운로드
- 추가로 CLI/API 실습도 원하면 Ollama까지 설치
- 공통 필수 조건: 행사 전 모델 다운로드와 실행 테스트까지 완료

권장안

- 8GB: 가능하면 다른 장비 권장. 꼭 가져오면 E2B만 시도, 속도 저하 감안
- 16GB: LM Studio + chat-ready / instruction-tuned `google/gemma-4-e4b`
- 32GB: LM Studio + chat-ready / instruction-tuned `google/gemma-4-e4b` 또는 `google/gemma-4-26b-a4b`
- 36GB: LM Studio + chat-ready / instruction-tuned `google/gemma-4-26b-a4b` 또는 `google/gemma-4-31b`
- ChromeOS: Ollama + `gemma4:e2b`

가장 비추천하는 조합

- 8GB Chromebook + 큰 모델
- Intel Mac + LM Studio
- 행사 당일 현장에서 처음 모델 다운로드

Ollama 태그 관련 주의

- 2026-04-27 기준 Ollama Gemma 4 태그 페이지에는 29개 태그가 표시됩니다.
- `gemma4:latest`는 현재 9.6GB, 128K 컨텍스트의 E4B 계열로 표시됩니다.
- `latest`는 향후 바뀔 수 있으므로, 사전 준비용으로는 `gemma4:e2b`, `gemma4:e4b`, `gemma4:26b`, `gemma4:31b`처럼 명시적 태그를 사용하세요.
- `*-cloud`, `*-mlx-*`, `*-mxfp8`, `*-nvfp4`, `*-q8_0` 같은 변형 태그는 목적을 알고 선택할 때만 사용하세요.
- 도구별 배포 포맷이 달라 다운로드 크기와 표시 용량은 서로 다를 수 있습니다.

문서 02

docs/02-windows-guide.md

Windows 참가자 빠른 준비 가이드

이 문서는 누구를 위한 문서인가요?

- Windows 노트북으로 이번 핸즈온에 참가하는 분
- Apple Silicon Mac이 아니라 일반적인 `Windows x64` 또는 `Windows ARM` 장비를 쓰는 분
- 어떤 도구를 설치해야 할지 빠르게 결정하고 싶은 분

먼저 결론

- Windows 참가자의 기본 권장 경로는 LM Studio입니다.
- CLI/API도 같이 써 보고 싶으면 Ollama를 추가로 설치하면 됩니다.
- `llama.cpp`는 고급 / 선택형 경로입니다.
- `Hermes Agent`는 Windows 네이티브가 아니라 WSL2 안에서만 공식 지원이므로, 모든 참가자에게 기본으로 권하지 않습니다.

즉 가장 현실적인 준비 순서는 아래입니다.

1. 기본 참가자: `LM Studio`
2. CLI까지 써 보고 싶은 참가자: `LM Studio + Ollama`
3. 고급 참가자: LM Studio 또는 llama.cpp + WSL2 안의 Hermes

어떤 도구를 설치하면 되나요?

경로 A: 가장 쉬운 기본 경로

설치:

- `LM Studio`

이 경로가 적합한 사람:

- GUI가 편한 분
- 로컬 모델을 한 번 실행해 보는 것이 우선인 분
- 핸즈온 현장에서 가장 안정적인 경로를 원하는 분

경로 B: 기본 + CLI/API 경로

설치:

- `LM Studio`
- `Ollama`

이 경로가 적합한 사람:

- GUI와 터미널 둘 다 써 보고 싶은 분
- 나중에 로컬 API를 붙여 보고 싶은 분

- 발표 도중 CLI 예시도 따라가 보고 싶은 분

경로 C: 고급 에이전트 경로

설치:

- `LM Studio` 또는 `llama.cpp`
- `WSL2`
- `Hermes Agent`는 WSL2 안에 설치

이 경로가 적합한 사람:

- `Hermes + Gemma 4` AI office 데모를 직접 해 보고 싶은 분
- 로컬 OpenAI 호환 서버에 에이전트를 연결해 보고 싶은 분

추천 조합

목표	추천 조합	비고
가장 쉬운 참가	`LM Studio`	기본 권장
GUI + CLI 둘 다	`LM Studio + Ollama`	가장 무난한 확장 경로
직접 서버 띄우기	`llama.cpp`	고급 경로
Hermes까지 실습	LM Studio 또는 llama.cpp + WSL2 + Hermes	선택형 고급 경로

메모리 기준 추천

메모리	추천 모델	권장 도구
8GB	E2B만 시도	LM Studio 우선
16GB	E4B	LM Studio 우선, 필요하면 Ollama 추가
32GB	E4B 또는 26B A4B	LM Studio, Ollama, llama.cpp 선택 가능
36GB+	26B A4B 또는 31B	고급 경로도 현실적

중요:

- 8GB Windows 노트북은 가능한 경우 다른 장비를 권장합니다.
- 16GB가 가장 무난한 최소 기준에 가깝습니다.
- 발표자 데모용으로는 32GB 이상이 훨씬 여유롭습니다.

Windows 참가자가 가장 먼저 해야 할 일

1. 기본 경로부터 끝내기

먼저 아래 중 하나 이상을 반드시 끝내세요.

- `LM Studio` 설치 + Gemma 4 실행 성공
- `Ollama` 설치 + `ollama run gemma4:e2b` 성공

- `llama.cpp` 설치 + `llama-server` 실행 성공

2. 모델도 미리 받아 두기

행사 당일에 모델을 처음 다운로드하지 않는 것이 중요합니다.

권장:

- 8GB: E2B
- 16GB: E4B
- 32GB+: E4B 또는 26B A4B

3. Hermes는 나중에

Windows 참가자는 `Hermes`를 가장 먼저 설치하기보다:

1. 모델이 실제로 한 번 도는지 먼저 확인
2. 그다음 필요하면 WSL2 + Hermes로 확장

이 순서가 훨씬 안전합니다.

WSL2는 꼭 필요한가요?

- `LM Studio`: 필요 없음
- `Ollama`: 필요 없음
- `llama.cpp`: 필요 없음
- `Hermes Agent`: 필요함

즉 Windows 참가자에게 WSL2는 Hermes를 쓸 때만 필수라고 이해하면 됩니다.

WSL2 설치는 PowerShell 관리자 권한으로 아래처럼 시작하면 됩니다.

```powershell
wsl --install -d Ubuntu
```

설치가 멈추면:

```powershell
wsl --install --web-download -d Ubuntu
```

설치 후 상태 확인:

```powershell
wsl --status
wsl --list --verbose
```

Windows + Hermes 권장 아키텍처

Windows에서 `Hermes`까지 하고 싶다면 가장 안전한 조합은 아래입니다.

```text
Windows native:
  LM Studio or llama.cpp server

WSL2:
  Hermes Agent
```

즉:

1. Windows에서 모델 서버 실행
2. WSL2 안에서 Hermes 실행
3. Hermes가 `http://127.0.0.1:PORT/v1` 또는 host endpoint에 연결

이 방식이 좋은 이유:

- WSL2 안에서 대용량 모델을 다시 받지 않아도 됨
- Windows GPU/GUI 앱 활용이 쉬움
- Hermes는 공식 지원 방식인 WSL2 안에서 돌릴 수 있음

설치 우선순위

기본 참가자

1. LM Studio 설치 가이드
2. 트러블슈팅 / 최종 체크 / 참고 링크

CLI도 해 보고 싶은 참가자

1. LM Studio 설치 가이드
2. Ollama 설치 가이드
3. 트러블슈팅 / 최종 체크 / 참고 링크

Hermes까지 해 보고 싶은 참가자

1. LM Studio 설치 가이드 또는 llama.cpp 설치 가이드
2. PowerShell 관리자 권한으로 WSL2 준비

```powershell
wsl --install -d Ubuntu
```

1. Ubuntu 실행

```powershell
wsl -d Ubuntu
```

1. Hermes Agent란 무엇인가
2. Hermes Agent 설치 가이드

절대 행사 당일에 처음 하면 안 되는 것

- WSL2 첫 설치
- 큰 모델 첫 다운로드
- `Hermes + WSL2 + local server`를 한 번도 테스트하지 않은 상태로 참석
- 8GB 노트북에서 큰 모델을 현장에서 바로 시도

행사 전 최종 체크리스트

아래 중 자신이 선택한 경로에 맞는 항목만 확인하면 됩니다.

LM Studio 경로

- 앱 설치 완료
- Gemma 4 다운로드 완료
- Chat 탭에서 1회 응답 생성 성공

Ollama 경로

- `ollama --version`
- `ollama pull gemma4:e2b` 또는 `ollama pull gemma4:e4b`
- `ollama run gemma4:e2b` 또는 `ollama run gemma4:e4b`

llama.cpp 경로

- `llama-cli --version`
- `llama-server --version`
- `llama-server` 또는 `llama-cli`로 1회 실행 성공
- 필요하면 `Invoke-RestMethod http://127.0.0.1:8080/v1/models` 확인

Hermes 경로

- WSL2 설치 완료
- PowerShell에서 `wsl --status` 확인
- WSL2 터미널에서 `hermes version`
- `hermes doctor`
- 로컬 서버 주소 연결 1회 성공

권장 한 줄 요약

- 대부분의 Windows 참가자: LM Studio만 준비해도 충분
- 조금 더 해 보고 싶으면: Ollama 추가
- AI office / agents까지 직접 하려면: WSL2 + Hermes는 선택형 고급 경로

문서 03

docs/03-memory-based-model-selection.md

메모리 기준으로 Gemma 4 모델 고르는 방법

이 문서에서 다루는 내용

- 공식 모델 스펙과 로컬 배포 크기 기준 메모리 판단법
- 노트북/데스크톱 메모리로 어떤 모델을 고르면 되는지
- 준비용 간단 계산식
- 8GB, 16GB, 32GB, 36GB 환경별 예시

먼저 알아둘 점

- 아래 표는 공식 모델 스펙과 LM Studio / Ollama에 표시되는 로컬 배포 크기를 함께 보는 기준입니다.
- 표시 크기는 다운로드 파일 또는 로컬 배포 패키지 기준이라, 실제 실행 중 최대 메모리 사용량과 항상 같지는 않습니다.
- 운영체제, 실행 도구, 브라우저, 긴 대화를 빠르게 이어가기 위해 저장하는 내부 메모리(KV cache), 긴 컨텍스트, 멀티모달 입력으로 인한 추가 메모리는 별도로 필요합니다.
- 따라서 실제 개인 노트북에서는 표의 숫자보다 여유를 더 잡아야 합니다.
- Apple Silicon Mac처럼 unified memory를 쓰는 환경에서는 전체 메모리를 하나의 풀처럼 보면 됩니다.
- Windows/Linux의 전용 GPU 노트북은 시스템 메모리(RAM)뿐 아니라 GPU 메모리(VRAM) 도 같이 확인해 주세요.

로컬 배포 크기와 컨텍스트 참고

2026-04-27 기준 LM Studio 모델 페이지와 Ollama Gemma 4 태그 페이지에 표시되는 기본 항목입니다.

모델	LM Studio 표시 크기	Ollama 기본 태그 표시 크기	컨텍스트
Gemma 4 E2B	4.20GB	7.2GB	128K
Gemma 4 E4B	5.90GB	9.6GB	128K
Gemma 4 26B A4B	17.00GB	18GB	256K
Gemma 4 31B	19.00GB	20GB	256K

읽는 법:

- LM Studio와 Ollama는 같은 모델이라도 배포 포맷과 양자화 방식이 달라 표시 크기가 다를 수 있습니다.
- Ollama의 `gemma4:latest`는 현재 E4B 계열로 표시되지만, `latest`는 바뀔 수 있으므로 명시적 태그를 사용하세요.
- 실행 중 메모리는 모델 크기에 더해 컨텍스트 길이, KV cache, 이미지 입력, 실행 도구 설정에 따라 늘어납니다.

26B A4B에서 A4B는 무엇인가요?

A4B는 Active 4B 로 이해하면 됩니다. 모델 전체는 26B 규모이지만, 답을 만들 때 실제로 계산에 참여하는 부분은 약 4B 규모라는 의미입니다. 공식 모델 카드 기준 수치는 `3.8B`입니다.

이런 구조를 `MoE(Mixture-of-Experts)`라고 부릅니다. 모델 안에 여러 "전문가" 부분이 있고, 답을 만들 때마다 필요한 일부 전문가만 골라 계산합니다.

중요한 점은 A4B가 메모리 사용량이 4B 모델만큼 작다는 뜻이 아니라는 것입니다. 26B A4B는 계산할 때 일부 전문가만 쓰지만, 빠른 추론을 위해 전체 26B 규모의 모델 파일을 메모리에 올려야 합니다. 그래서 로컬 배포 크기도 E4B보다 훨씬 큽니다.

메모리 계획 포인트

- E2B, E4B는 이름에 보이는 크기보다 모델 파일이 더 클 수 있습니다. 작은 모델을 효율적으로 쓰기 위한 추가 구조가 포함되어 있기 때문입니다.
- 26B A4B는 계산할 때 일부만 쓰는 구조이지만, 전체 모델 파일을 메모리에 올려야 하므로 메모리 요구량이 4B 모델처럼 작아지지 않습니다.
- 긴 컨텍스트는 이전 대화 내용을 빠르게 이어가기 위한 내부 저장 공간 때문에 메모리를 더 사용합니다.
- fine-tuning은 inference보다 훨씬 더 많은 메모리가 필요합니다.

준비용 간단 계산식

아래 계산식은 공식 수치가 아니라 준비용 간단 추정식입니다.

1. 사용 가능 메모리 계산

```text
사용 가능 메모리 = 전체 메모리 - 시스템/앱 여유분 - 컨텍스트 여유분
```

권장 기본값:

- 시스템/앱 여유분: 4~6GB
- 컨텍스트 여유분: 1~3GB

여유 있게 잡은 권장값:

```text
사용 가능 메모리 ≈ 전체 메모리 - 6GB - 2GB
```

즉,

```text
사용 가능 메모리 ≈ 전체 메모리 - 8GB
```

이 값보다 모델의 대략적 로드 메모리가 작아야, 현장에서 비교적 안정적으로 쓸 가능성이 높습니다.

가장 쉬운 선택 공식

LM Studio 기본 경로를 기준으로 준비할 때는 아래처럼 생각하면 됩니다.

```text
권장 전체 메모리 ≈ 로컬 모델 표시 크기 + 8GB
```

예시:

- E2B: 4.2GB + 8GB ≈ 12.2GB

- E4B: 5.9GB + 8GB ≈ 13.9GB
- 26B A4B: 17GB + 8GB ≈ 25GB
- 31B: 19GB + 8GB ≈ 27GB

Ollama 기본 태그는 같은 모델이라도 표시 크기가 더 클 수 있습니다. 특히 16GB 장비에서 `gemma4:e4b`가 버겁게 느껴지면 `gemma4:e2b`로 낮추세요.

이 계산은 "행사에서 너무 빡빡하지 않게 쓰는 기준"입니다. 즉:

- E2B는 8GB에서는 거의 최소치이며 속도 저하 가능
- E4B는 16GB에서 현실적
- 26B A4B, 31B는 32GB 이상에서 현실적

RAM만 보면 되나요?

항상 그렇지는 않습니다.

- Apple Silicon Mac: unified memory 구조라서, 보통은 이 문서의 "전체 메모리 기준 계산"을 그대로 적용해도 됩니다.
- Windows/Linux + 전용 GPU: GPU 가속을 쓸 때는 시스템 RAM과 별개로 VRAM이 부족할 수 있습니다.
- 예를 들어 시스템 메모리가 32GB여도 VRAM이 작으면 큰 모델은 GPU에 충분히 못 올라가서 속도가 느려지거나, 일부 오프로딩으로 체감이 달라질 수 있습니다.
- 전용 GPU 사양이 애매하면 더 작은 모델(E2B/E4B) 을 고르세요.

메모리별로 어떻게 고르면 되나요?

8GB

간단 계산:

```text
사용 가능 메모리 ≈ 8GB - 8GB = 0GB
```

실제로는 아주 빡빡합니다.

해석:

- E2B도 운영체제와 앱이 같이 뜬 상태에서는 여유가 거의 없습니다.
- 따라서 8GB는 E2B만 시도하고, 속도 저하를 감안해 주세요.
- Windows/macOS에서 LM Studio 공식 권장은 16GB 이상이라, 8GB는 "무난한 기본 사양"이 아닙니다.
- 특히 8GB Apple Silicon Mac이나 8GB 일반 노트북에서는 E2B/E4B 실행 시 전체 시스템이 매우 느려지거나 일시적으로 멈춘 것처럼 보일 수 있습니다.

권장:

- E2B만 준비
- 가능하면 16GB 이상 장비 사용
- 브라우저 탭 최소화
- 다른 앱 종료

- 가능하면 전원 연결

16GB

간단 계산:

```text
사용 가능 메모리 ≈ 16GB - 8GB = 8GB
```

해석:

- E4B는 LM Studio 기본 경로에서 현실적인 선택
- E2B는 더 여유 있음
- 26B A4B, 31B는 사실상 어렵습니다

권장:

- 기본: E4B
- 실패 시: E2B

32GB

간단 계산:

```text
사용 가능 메모리 ≈ 32GB - 8GB = 24GB
```

해석:

- E4B는 매우 여유 있음
- 26B A4B는 현실적
- 31B도 가능 범위에 들어오지만, 긴 컨텍스트와 다른 앱까지 생각하면 여유가 줄어듭니다

권장:

- 안정성 우선: E4B
- 성능 우선: 26B A4B
- 실험적 상위 옵션: 31B

36GB

간단 계산:

```text
사용 가능 메모리 ≈ 36GB - 8GB = 28GB
```

해석:

- 26B A4B, 31B 모두 현실적

- 31B도 가능성이 높지만, 속도와 반응성은 26B A4B가 더 나은 경우가 있을 수 있습니다

권장:

- 균형형: 26B A4B
- 품질 우선: 31B

GGUF / MLX / 도구별로 이 표를 어떻게 보면 되나요?

- Ollama / LM Studio GGUF: 보통 4비트~8비트 계열 로컬 모델을 많이 쓰지만, 도구별 표시 크기와 실제 메모리 사용량은 다를 수 있습니다.
- MLX: Apple Silicon에서 잘 맞는 선택지이지만, 실제 메모리 사용량은 모델 변환 방식과 정밀도에 따라 달라질 수 있습니다.
- 즉, 위 표는 준비용 기준점이고, 실제 파일 크기와 런타임 메모리는 도구별로 조금씩 다릅니다.

한 줄 정리

- 8GB: E2B만 시도, 속도 저하 감안
- 16GB: E4B
- 32GB: E4B 또는 26B A4B
- 36GB: 26B A4B 또는 31B

참고 링크

- Gemma 4 model overview
- Gemma 4 model card
- LM Studio Gemma 4 models
- Ollama Gemma 4 tags
- Google Korean blog: Gemma 4 소개

문서 04

docs/04-gguf-mlx-llamacpp-explainer.md

GGUF, MLX, llama.cpp 개념 설명

이 문서는 누구를 위한 문서인가요?

- GGUF, MLX, llama.cpp가 무엇인지 헷갈리는 참가자
- LM Studio에서 왜 어떤 모델은 GGUF, 어떤 모델은 MLX로 보이는지 알고 싶은 참가자
- Apple Silicon Mac과 Windows/Linux에서 어떤 실행 경로가 다른지 이해하고 싶은 참가자

먼저 아주 짧게 정리

- GGUF: 모델 파일 포맷
- llama.cpp: 로컬 LLM 실행 엔진 / 런타임
- MLX: Apple Silicon용 머신러닝 프레임워크

즉:

- GGUF는 파일
- llama.cpp는 그 파일을 실행하는 쪽
- MLX는 Apple Silicon에서 모델을 실행하는 다른 경로

가장 중요한 오해 하나

GGUF, MLX, llama.cpp는 같은 종류의 단어가 아닙니다.

- GGUF는 파일 포맷입니다.
- llama.cpp는 추론 엔진입니다.
- MLX는 Apple의 프레임워크이자 그 위에서 돌아가는 실행 생태계입니다.

예를 들면:

- 같은 Gemma 4 모델도 GGUF 버전이 있을 수 있고
- Apple Silicon에서는 MLX 버전이 따로 있을 수 있습니다.

즉, 같은 모델이 여러 실행 포맷/런타임으로 제공될 수 있다고 이해하면 됩니다.

GGUF란?

GGUF 공식 문서 기준:

- GGUF는 GGML 및 GGML 기반 실행기에서 추론용 모델을 저장하는 파일 포맷입니다.
- 빠른 로딩, mmap 친화성, 단일 파일 배포, 메타데이터 포함을 목표로 설계되었습니다.
- 보통 모델은 처음에 PyTorch 같은 프레임워크에서 학습되고, 이후 로컬 추론용으로 GGUF로 변환되어 배포됩니다.

쉽게 말하면:

- Hugging Face 원본 체크포인트를

- 로컬 PC에서 다루기 쉬운 형태로 바꾼
- 로컬 실행용 파일 포맷

이라고 보면 됩니다.

참가자 관점에서 중요한 점:

- Windows/Linux에서 로컬 LLM을 돌릴 때 가장 흔히 보는 포맷 중 하나가 `GGUF`입니다.
- LM Studio, Ollama, llama.cpp 같은 도구와 함께 자주 등장합니다.
- `Q4`, `Q5`, `Q8` 같은 양자화 표기가 붙은 GGUF 모델을 많이 보게 됩니다.

llama.cpp란?

llama.cpp 공식 README 기준:

- llama.cpp는 C/C++ 기반 LLM inference 엔진입니다.
- 최소한의 설정으로 다양한 하드웨어에서 로컬/클라우드 추론을 할 수 있도록 만드는 것이 목표입니다.
- Apple Silicon, x86, NVIDIA GPU, AMD GPU 등 여러 백엔드를 지원합니다.
- OpenAI 호환 API 서버(`llama-server`)도 제공할 수 있습니다.

이름 때문에 오해하기 쉬운데:

- `llama.cpp`는 Llama 모델만 실행하는 도구가 아닙니다.
- 실제로 Gemma, Qwen, Mistral 등 `llama.cpp` 호환 모델도 많이 실행합니다.

참가자 관점에서 쉽게 말하면:

- GGUF 파일을 실제로 돌려주는 대표적인 실행 엔진

입니다.

정리하면:

- LM Studio는 문서상 GGUF 모델을 `llama.cpp`로 실행

라고 이해하면 큰 흐름을 잡는 데 도움이 됩니다.

MLX란?

Apple MLX 공식 README 기준:

- MLX는 Apple Silicon용 머신러닝 프레임워크입니다.
- CPU와 GPU를 아우르는 unified memory 구조를 활용합니다.
- Python, C++, C, Swift API를 제공합니다.

LLM 관점에서 더 직접적인 것은 `mlx-lm` 입니다.

Apple `mlx-lm` 공식 README 기준:

- `mlx-lm`은 Apple Silicon에서 LLM을 실행하고 fine-tuning 하는 패키지입니다.
- Hugging Face 모델을 불러오고
- 양자화하고

- MLX 형식으로 변환하고
- LoRA/QLoRA fine-tuning도 할 수 있습니다.

즉, 참가자 관점에서는:

- `MLX`는 Apple Silicon 전용 경로
- MLX 모델은 Apple Silicon에서 잘 맞는 실행 방식

으로 이해하면 됩니다.

GGUF와 MLX는 어떻게 다른가요?

항목	GGUF	MLX
성격	로컬 추론용 모델 파일 포맷	Apple Silicon용 프레임워크/실행 생태계
대표 실행 경로	`llama.cpp`, LM Studio, Ollama	MLX, `mlx-lm`, LM Studio(Apple Silicon)
운영체제	macOS, Windows, Linux 전반	사실상 Apple Silicon 중심
참가자 관점	가장 흔한 범용 로컬 포맷	Mac Apple Silicon에서만 고려할 선택지

핵심:

- `GGUF`는 더 범용적입니다.
- `MLX`는 Apple Silicon에 특화된 선택지입니다.

LM Studio에서는 왜 둘 다 보이나요?

LM Studio 공식 문서 기준:

- LM Studio는 macOS, Windows, Linux에서 `llama.cpp` 기반 GGUF 모델 실행을 지원합니다.
- Apple Silicon Mac에서는 Apple의 MLX 실행 경로도 지원합니다.

그래서 LM Studio에서:

- Windows/Linux에서는 주로 `GGUF`
- Apple Silicon Mac에서는 `GGUF`와 `MLX` 둘 다

보일 수 있습니다.

참가자 입장에서는 무엇을 선택하면 되나요?

이번 핸즈온에서는:

- Windows/Linux: `GGUF` 중심으로 생각하면 됩니다.
- Apple Silicon Mac: GGUF 기본, MLX는 옵션

즉:

- 개념이 헷갈리면 그냥 GGUF를 기본 선택지로 보면 됩니다.
- Apple Silicon 사용자만 필요할 때 `MLX`를 추가로 이해하면 충분합니다.

왜 GGUF가 행사 기본 설명에 더 자주 나오나요?

이유는 단순합니다.

- 참가자 환경이 macOS만 있는 것이 아니고
- Windows/Linux/ChromeOS까지 같이 봐야 하며
- LM Studio, Ollama, llama.cpp 같은 로컬 도구에서 범용적으로 이해하기 쉬운 축이 `GGUF`이기 때문입니다.

그래서 이번 핸즈온에서는:

- `GGUF`를 기본 개념
- `MLX`를 Apple Silicon 추가 옵션

으로 이해하면 됩니다.

llama.cpp는 행사에서 왜 알아두면 좋나요?

참가자가 꼭 직접 설치할 필요는 없지만, 아래를 이해하는 데 도움이 됩니다.

- 왜 GGUF 모델이 여러 로컬 도구에서 공통으로 보이는지
- 왜 `Q4`, `Q5`, `Q8` 같은 양자화 모델이 자주 나오는지
- 왜 OpenAI 호환 로컬 서버를 만들 수 있는지

즉, llama.cpp는 로컬 LLM 생태계에서 기초 엔진 역할을 하는 경우가 많습니다.

한 줄 정리

- `GGUF`: 범용 로컬 모델 파일
- `llama.cpp`: 그 파일을 돌려주는 대표 엔진
- `MLX`: Apple Silicon에서 쓰는 별도 최적화 경로

이번 핸즈온 기준 추천 해석

- 개념이 어렵다면 GGUF 기본으로 이해해도 충분합니다.
- Apple Silicon Mac 사용자만 MLX는 선택 사항으로 추가 이해하면 됩니다.
- `llama.cpp`는 참가자가 직접 만지는 도구가 아니더라도, LM Studio/GGUF를 이해하는 배경지식으로 알면 좋습니다.

같이 보면 좋은 문서

- LM Studio 설치 가이드
- Apple Silicon + MLX 안내
- 메모리 기준으로 Gemma 4 모델 고르는 방법

참고 링크

- GGUF spec (ggml)
- llama.cpp README
- Apple MLX README

- Apple MLX LM README
- LM Studio Docs

문서 05

docs/05-lm-studio-setup.md

LM Studio 설치 가이드

먼저 알아둘 점

- 이번 세션은 개발자, 비개발자 모두 참여 가능한 세션이므로, 하나만 설치한다면 LM Studio를 기본 권장합니다.
- LM Studio는 macOS, Windows, Linux를 지원합니다.
- macOS는 Apple Silicon만 지원합니다.
- macOS 앱은 공식 문서 기준 13.4 이상이 필요하며, MLX 모델은 macOS 14.0 이상이 필요합니다.
- Windows는 x64와 ARM을 지원합니다.
- Linux는 Ubuntu 20.04+ AppImage가 공식 기준입니다.
- 모델 다운로드 후에는 완전 오프라인 사용이 가능합니다.
- Apple Silicon Mac에서는 GGUF를 기본으로 사용해도 되고, 필요하면 MLX 모델도 선택할 수 있습니다.
- 이번 핸즈온은 채팅/실습/에이전트 흐름이 중심이므로, 가능하면 instruction-tuned(chat-ready) 모델을 선택하세요.
- LM Studio에서는 모델 이름이 `...-it` 로 그대로 보일 수도 있고, `google/gemma-4-e4b`처럼 더 짧게 보일 수도 있으므로 설명이나 모델 카드에서 chat-ready / instruct 계열인지 함께 확인해 주세요.

Windows

권장 대상:

- Windows x64 또는 ARM
- RAM 16GB 이상 권장

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. LM Studio 다운로드
2. Windows용 설치 파일을 받아 실행합니다.
3. 앱을 한 번 실행합니다.
4. 필요 시 런타임을 설치하거나 업데이트합니다. 단축키: `Ctrl + Shift + R`
5. `Discover` 탭에서 `gemma 4`를 검색합니다.
6. 아래 중 하나를 다운로드합니다.

- `google/gemma-4-e2b` 계열의 chat-ready / instruction-tuned 항목
- `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목

1. `Chat` 탭으로 이동해 모델을 메모리에 로드합니다.
2. 테스트 프롬프트를 입력합니다.

예시 프롬프트:

```text
안녕하세요. Build with AI Seoul 2026 핸즈온 테스트입니다. 한 줄로 자기소개해
주세요.
```

선택 사항:

- 터미널에서 `lms --help`를 실행해 CLI까지 확인할 수 있습니다.
- 단, `lms`는 LM Studio를 최소 1회 실행한 뒤 사용하는 것이 안전합니다.
- Windows에서는 PowerShell 기준으로 아래처럼 확인하면 됩니다.

```powershell
lms --help
```

macOS

권장 대상:

- Apple Silicon (M1/M2/M3/M4)
- macOS 13.4 이상
- MLX 모델까지 활용하려면 macOS 14.0 이상

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. LM Studio 다운로드
2. macOS용 앱을 설치합니다.
3. 앱을 실행합니다.
4. 필요 시 런타임을 설치하거나 업데이트합니다. 단축키: `Cmd + Shift + R`
5. `Discover` 탭에서 `gemma 4`를 검색합니다.
6. 아래 중 하나를 다운로드합니다.

- `google/gemma-4-e2b` 계열의 chat-ready / instruction-tuned 항목
- `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목

1. `Chat` 탭에서 모델을 로드하고 테스트합니다.

권장 메모:

- LM Studio 공식 문서는 macOS에서 16GB+ RAM 권장입니다.
- LM Studio 앱 자체는 macOS 13.4 이상을 지원하고, MLX 모델은 macOS 14.0 이상이 필요합니다.
- 행사 준비 기준으로는 최신 macOS를 권장합니다.
- 8GB Mac도 가능할 수 있지만, 작은 모델과 작은 컨텍스트만 권장합니다.
- Apple Silicon에서도 GGUF 기반 fine-tuned 모델을 사용하는 것은 대체로 가능하지만, 양자화 방식, 채팅 템플릿, 멀티모달 지원 여부에 따라 모델별 확인이 필요합니다.
- MLX는 선택 사항이며, 행사 준비 기준으로는 GGUF만 사용해도 충분합니다.

- `GGUF`, `MLX`, `llama.cpp` 개념이 헷갈리면 GGUF, MLX, llama.cpp 개념 설명 문서를 먼저 확인해 주세요.
- Apple Silicon 사용자는 필요하면 Apple Silicon + MLX 안내도 함께 확인해 주세요.
- LM Studio를 `opencode` 같은 코딩 에이전트 도구와 연결해 보고 싶다면 OpenCode 설치 및 LM Studio 연동 가이드도 함께 확인해 주세요.
- Windows 사용자는 LM Studio CLI 예시를 볼 때 `bash` 대신 PowerShell로 따라오시면 됩니다.

Linux (Ubuntu)

권장 대상:

- Ubuntu 20.04 이상
- x64 또는 ARM64

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. LM Studio 다운로드
2. Linux용 AppImage를 다운로드합니다.
3. 터미널에서 실행 권한을 부여합니다.

```bash
chmod +x LM-Studio-*.AppImage
```

1. AppImage를 실행합니다.

```bash
./LM-Studio-*.AppImage
```

1. 앱이 실행되면 `Discover` 탭에서 `gemma 4`를 검색합니다.
2. `google/gemma-4-e2b` 또는 `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목을 다운로드합니다.
3. `Chat` 탭에서 모델을 로드하고 테스트합니다.

참고:

- 공식 문서는 Linux 배포 형식을 AppImage로 안내합니다.
- 위 AppImage 실행 절차는 그 배포 형식에 맞춘 일반적인 설치 흐름입니다.
- Ubuntu 22.04보다 최신 버전은 공식 문서상 충분히 테스트되지 않았다고 안내되어 있습니다.

ChromeOS

ChromeOS에서는 LM Studio를 권장하지 않습니다.

이유:

- LM Studio Linux 공식 지원 기준은 Ubuntu 20.04+ AppImage입니다.
- ChromeOS Linux 개발 환경은 Debian 기반입니다.

- ChromeOS Linux 환경에서는 로컬 LLM 실행에 필요한 전용 GPU/VRAM 기반 compute 가속을 기대하기 어렵습니다.

즉, ChromeOS에서 LM Studio는 공식 권장 경로 밖이며, 행사 당일 설치 이슈가 생길 가능성이 있습니다.

ChromeOS 참가자 권장안:

1. Ollama로 준비
2. 가능하면 Windows 또는 macOS 노트북 추가 준비

문서 06

docs/06-ollama-setup.md

Ollama 설치 가이드

이 문서는 언제 보면 되나요?

- LM Studio 외에 CLI/API 방식도 함께 준비하고 싶을 때
- ChromeOS 참가자일 때
- Intel Mac처럼 LM Studio 데스크톱 앱이 맞지 않는 환경일 때

Windows

권장 대상:

- Windows 10 22H2 이상
- NVIDIA 또는 AMD GPU가 있으면 더 좋음

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. Ollama Windows 다운로드
2. 가장 쉬운 방법은 설치 프로그램(`OllamaSetup.exe`)을 사용하는 것입니다.
3. 설치 후 PowerShell 또는 명령 프롬프트를 열고 아래 명령으로 확인합니다.

```powershell
ollama --version
```

1. 메모리에 맞는 Gemma 4 모델을 미리 받습니다.

```powershell
ollama pull gemma4:e2b
```

또는

```powershell
ollama pull gemma4:e4b
```

1. 실행 테스트:

```powershell
ollama run gemma4:e2b
```

권장 메모:

- Ollama Windows 설치는 공식 문서 기준 관리자 권한이 필수는 아닙니다.
- 모델은 기본적으로 사용자 홈 디렉터리 아래에 저장됩니다.

- 설치 후 Ollama는 백그라운드에서 실행되고, 로컬 API는 기본적으로 `http://localhost:11434`에서 동작합니다.
- Windows에서는 명령 프롬프트(CMD) 도 가능하지만, 이 가이드는 PowerShell 기준입니다.
- Ollama는 `gemma4:e2b`, `gemma4:e4b` 같은 런타임 태그를 사용하므로, 사용자가 따로 `-it` 접미사를 고르지 않아도 됩니다. 핸즈온에서는 이 태그들을 chat-ready 실행 경로로 이해하면 됩니다.

Gemma 4 태그 선택

2026-04-27 기준 Ollama Gemma 4 태그 페이지에는 29개 태그가 표시됩니다.

사전 준비에서는 목적이 명확한 로컬 기본 태그를 사용하세요.

- 8GB: `gemma4:e2b`
- 16GB: `gemma4:e4b`, 버거우면 `gemma4:e2b`
- 32GB 이상: `gemma4:26b`
- 36GB 이상: `gemma4:26b` 또는 `gemma4:31b`

주의:

- `gemma4:latest`는 현재 E4B 계열로 표시되지만, 나중에 바뀔 수 있습니다.
- `*-cloud` 태그는 로컬에 모델을 받아 두는 경로가 아니므로, 사전 준비 기본값으로 쓰지 마세요.
- `*-mlx-*`, `*-mxfp8`, `*-nvfp4`, `*-q8_0` 같은 변형 태그는 정밀도와 실행 목적을 알고 선택할 때만 사용하세요.

macOS

권장 대상:

- macOS 14 Sonoma 이상
- Apple Silicon 권장
- Intel Mac도 가능하지만 CPU 전용

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. Ollama macOS 다운로드
2. `ollama.dmg`를 열고 Ollama 앱을 `Applications` 폴더로 옮깁니다.
3. 앱을 처음 실행합니다.
4. 터미널에서 설치 확인:

```bash
ollama --version
```

1. 메모리에 맞는 모델을 미리 받습니다.

```bash
ollama pull gemma4:e2b
```

또는

```bash
ollama pull gemma4:e4b
```

1 실행 테스트:

```bash
ollama run gemma4:e2b
```

권장 메모:

- 첫 실행 시 CLI 경로 연결을 요청하면 허용하세요.
- Apple Silicon에서는 CPU/GPU를 함께 활용하기 쉽고, 행사 진행에도 유리합니다.

Linux (Ubuntu 등)

권장 대상:

- 일반 Linux 노트북 또는 데스크톱
- NVIDIA/AMD GPU가 있으면 더 유리
- ARM64 장치도 가능하지만 행사 안정성 기준으로는 x64가 더 무난

설치 순서:

1 터미널을 엽니다.

2 Ollama를 설치합니다.

```bash
curl -fsSL https://ollama.com/install.sh | sh
```

1 설치 확인:

```bash
ollama -v
```

1 모델 다운로드:

```bash
ollama pull gemma4:e2b
```

또는

```bash
ollama pull gemma4:e4b
```

1 실행 테스트:

```bash
ollama run gemma4:e2b
```

추가 메모:

- AMD GPU가 있으면 ROCm 패키지를 별도로 설치할 수 있습니다.
- ARM64 장치에서는 ARM64용 패키지를 써야 합니다.
- 서버처럼 계속 띄워둘 경우 `ollama serve`와 systemd 서비스 설정도 가능합니다.

ChromeOS

ChromeOS는 Ollama만 권장합니다. 다만 이 경로는 안정성이 낮을 수 있으며, Windows/macOS보다 느릴 수 있습니다.

중요 제한:

- Chromebook에서 Linux 개발 환경이 켜져 있어야 합니다.
- 회사/학교 지급 기기에서는 Linux 사용이 막혀 있을 수 있습니다.
- ChromeOS Linux 환경에서는 Ollama/Gemma 4 같은 로컬 LLM 실행에서 전용 GPU/VRAM 기반 compute 가속을 기대하기 어렵습니다.
- 따라서 ChromeOS에서는 E2B 위주로 준비하는 것을 권장합니다.

설치 순서:

1. ChromeOS에서 Linux 개발 환경을 켭니다. 경로: 설정 > About ChromeOS > Developers > Linux development environment > Set up
2. Linux 터미널이 열리면 패키지를 업데이트합니다.

```bash
sudo apt-get update && sudo apt-get dist-upgrade
```

1. Ollama를 설치합니다.

```bash
curl -fsSL https://ollama.com/install.sh | sh
```

1. 설치 확인:

```bash
ollama -v
```

1. 모델 다운로드:

```bash
ollama pull gemma4:e2b
```

1 실행 테스트:

```bash
ollama run gemma4:e2b
```

ChromeOS 권장 메모:

- 8GB Chromebook: 가능하면 다른 기기를 권장, 꼭 써야 한다면 E2B만 시도
- 16GB 이상 Chromebook: E2B 권장, E4B는 속도가 매우 느릴 수 있음

문서 07

docs/07-llamacpp-setup.md

llama.cpp 설치 가이드

이 문서는 누구를 위한 문서인가요?

- `GGUF + llama.cpp`로 Gemma 4를 로컬에서 돌려 보고 싶은 참가자
- `llama-server`를 띄워 `Hermes`, `OpenCode`, `Pi` 같은 에이전트와 연결하고 싶은 참가자
- `LM Studio` 대신 CLI 기반 local server 경로를 선호하는 참가자

먼저 결론

- `llama.cpp`는 이번 세션의 고급 / 선택형 경로입니다.
- 가장 쉬운 설치 경로는 pre-built package 설치입니다.
- 공식 문서 기준으로:
- macOS / Linux: `brew install llama.cpp`
- Windows: `winget install llama.cpp`
- 이번 핸즈온에서는 소스 빌드보다 pre-built 설치를 우선 권장합니다.

왜 llama.cpp를 쓰나요?

- `GGUF` 모델을 직접 로컬에서 돌릴 수 있습니다.
- `llama-server`로 OpenAI 호환 API 서버를 띄울 수 있습니다.
- `Hermes` 같은 에이전트가 로컬 서버 주소에 붙기 쉽습니다.

권장 설치 경로

macOS / Linux

```bash
brew install llama.cpp
```

설치 확인:

```bash
llama-cli --version
llama-server --version
```

Windows

PowerShell:

```powershell
winget install llama.cpp
```

설치 확인:

```powershell
llama-cli --version
llama-server --version
```

모델 파일 경로에 공백이 있다면 큰따옴표로 감싸 주세요.

대안 설치 경로

공식 문서에는 아래도 있습니다.

- pre-built binaries 다운로드
- Docker
- 소스 빌드

핸즈온 참가자는 일단 패키지 매니저 설치를 우선 사용하세요.

소스 빌드는 언제 필요한가요?

- GPU backend를 직접 선택해 빌드해야 할 때
- 최신 기능이 패키지보다 먼저 필요할 때
- 배포 패키지 대신 특정 빌드 옵션을 직접 제어하고 싶을 때

공식 CPU build 기본 예시는 아래입니다.

```bash
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
cmake -B build
cmake --build build --config Release
```

이번 사전 준비 가이드에서는 소스 빌드를 기본 경로로 권장하지 않습니다.

가장 간단한 실행 예시

로컬 GGUF 파일 사용

```bash
llama-cli -m /path/to/model.gguf
```

Windows PowerShell:

```powershell
llama-cli -m "C:\path\to\model.gguf"
```

Hugging Face에서 바로 다운로드해서 실행

```bash
llama-cli -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

Windows PowerShell:

```powershell
llama-cli -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

OpenAI 호환 API 서버 실행

```bash
llama-server -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

Windows PowerShell:

```powershell
llama-server -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

또는 이미 내려받은 GGUF 파일이 있다면:

```bash
llama-server -m /path/to/model.gguf --host 127.0.0.1 --port 8080
```

Windows PowerShell:

```powershell
llama-server -m "C:\path\to\model.gguf" --host 127.0.0.1 --port 8080
```

기본 Web UI는 보통 아래에서 열 수 있습니다.

```text
http://localhost:8080
```

chat completions endpoint는 아래입니다.

```text
http://localhost:8080/v1/chat/completions
```

Gemma 4와 함께 쓸 때

이번 핸즈온에서는 `Gemma 4 GGUF`를 `llama.cpp`로 띄우고, 필요하면 그 위에 `Hermes` 같은 에이전트를 연결하는 구성이 가능합니다.

예시:

```bash
llama-server -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

또는:

```bash
llama-server -m /absolute/path/to/gemma-4-E4B-it-Q4_K_M.gguf --host 127.0.0.1 --port 8080
```

행사 전 최종 확인

아래가 되면 준비 완료입니다.

llama.cpp 준비 완료 기준

macOS / Linux:

```bash
llama-cli --version
llama-server --version
```

Windows PowerShell:

```powershell
llama-cli --version
llama-server --version
```

그리고 아래 중 하나 이상:

1. `llama-cli`로 한 번 답변 생성 성공
2. `llama-server` 실행 성공
3. `curl http://127.0.0.1:8080/v1/models` 확인 성공

예시:

```bash
curl http://127.0.0.1:8080/v1/models
```

Windows PowerShell:

```powershell
Invoke-RestMethod http://127.0.0.1:8080/v1/models
```

주의할 점

- `llama.cpp`만 설치해도 모델 파일은 자동으로 준비되지 않습니다.
- 대용량 모델은 다운로드 시간이 길 수 있으므로 행사 전에 미리 받아 두세요.

- GGUF 경로를 직접 관리해야 할 수 있습니다.
- Windows에서는 `llama.cpp` 자체는 설치 가능하지만, 그 위에 Hermes를 붙일 때는 Hermes가 WSL2 안에서 돌아가야 한다는 점을 별도로 이해해야 합니다.

누구에게 권하나요?

- CLI에 익숙한 참가자
- LM Studio보다 더 직접적인 local server 구성을 원하는 참가자
- Hermes + Gemma 4를 직접 묶어 보고 싶은 참가자

공식 참고 링크

- llama.cpp README
- llama.cpp Install Guide
- llama.cpp Build Guide

문서 08

docs/08-apple-silicon-mlx.md

Apple Silicon + MLX 안내

이 문서는 누구를 위한 문서인가요?

- Apple Silicon Mac(M1/M2/M3/M4) 사용자
- LM Studio에서 GGUF 말고 MLX 경로도 이해하고 싶은 참가자
- MLX 기반 파생 모델이나 fine-tuned 모델이 있는지 궁금한 참가자

MLX란?

- MLX는 Apple Silicon용 머신러닝 프레임워크입니다.
- Apple의 `mlx-lm` 프로젝트는 Apple Silicon에서 LLM 실행뿐 아니라 fine-tuning(LoRA/QLoRA 포함)도 지원합니다.
- LM Studio 공식 문서도 Apple Silicon Mac에서 Apple의 MLX를 지원한다고 안내합니다.
- `GGUF`, `MLX`, `llama.cpp` 차이를 먼저 보고 싶다면 GGUF, MLX, llama.cpp 개념 설명 문서를 함께 확인해 주세요.

행사 준비에서 MLX는 어떻게 보면 되나요?

- Apple Silicon 사용자는 LM Studio에서 GGUF뿐 아니라 MLX 모델도 선택할 수 있습니다.
- Gemma 4도 LM Studio 모델 페이지 기준 GGUF와 MLX로 제공됩니다.
- 다만 이번 핸즈온 참가 준비 관점에서는 MLX 자체가 필수는 아닙니다.
- Apple Silicon에서도 GGUF 기반 fine-tuned 모델을 대체로 사용할 수 있지만, 양자화 방식, 채팅 템플릿, 멀티모달 지원 여부에 따라 모델별 호환성과 품질 확인이 필요합니다.
- 즉, macOS 사용자는 GGUF 기본 + 필요하면 MLX 추가 고려 정도로 이해하면 됩니다.
- 가장 중요한 것은 행사 전에 사용할 모델을 미리 다운로드하고 실행 확인을 끝내는 것입니다.

MLX 기반 Gemma 4 모델 예시

Hugging Face에는 아래와 같은 MLX 형식 Gemma 4 모델이 올라와 있습니다.

- `mlx-community/gemma-4-e2b-it-8bit`
- `mlx-community/gemma-4-e4b-it-8bit`

이 모델들은 Hugging Face 카드 기준으로 각각 원본 Google Gemma 4 체크포인트를 MLX 형식으로 변환한 모델입니다.

MLX 기반 fine-tuned 모델도 있나요?

있습니다.

- Apple의 `mlx-lm`은 Gemma 계열을 포함한 여러 모델 패밀리에 대해 LoRA/QLoRA fine-tuning을 지원합니다.
- Hugging Face에는 MLX 기반 파생 모델이나 fine-tuned 모델도 존재합니다.
- 예를 들어 `emanubiz/gemma4-E4B-opus-finetuned` 같은 모델은 `mlx-community/gemma-4-e4b-it-4bit`를 바탕으로 LoRA fine-tuning 한 사례입니다.

즉, Apple Silicon 환경에서는 MLX로 실행 가능한 Gemma 파생 모델이나 fine-tuned 모델을 활용할 수 있습니다.

`mlx-community` 관련 주의

- `mlx-community`는 공식 Google 배포처가 아니라 Hugging Face 커뮤니티 업로드/변환 흐름을 사용하는 조직입니다.
- 따라서 `mlx-community` 계열 모델은 공식 Google 원본이나 Ollama, LM Studio 허브의 기본 항목과 비교해 업데이트나 패치 반영 시점이 늦을 수 있습니다.
- 커뮤니티 모델은 변환 방식, 채팅 템플릿, 툴 호출, 멀티모달 동작이 제각각일 수 있습니다.

권장:

- 행사 당일에는 공식 Gemma 4 기본 모델(E2B/E4B) 사용을 우선 권장합니다.
- MLX 파생 모델, 커뮤니티 변환 모델, 개인 fine-tuned 모델은 추가 실험용으로만 권장합니다.

Apple Silicon 사용자 권장안

- 하나만 설치한다면 LM Studio
- 메모리 8GB면 E2B만 시도, 속도 저하 감안
- 메모리 16GB면 E4B
- 행사 준비는 공식 기본 모델 우선
- GGUF로 준비해도 충분함
- GGUF fine-tuned 모델은 대체로 가능하지만, 모델별 호환성과 품질 확인 필요
- MLX는 선택 사항

8GB Apple Silicon Mac 추가 안내:

- 8GB 장비는 E2B에서도 속도가 많이 느리거나 시스템이 잠깐 멈춘 것처럼 보일 수 있습니다.
- E4B는 실행되더라도 행사 당일 안정성이 떨어질 수 있습니다.
- 가능하면 16GB 이상 메모리 장비를 권장합니다.
- 8GB 장비로 참석하는 경우에는 작은 모델, 짧은 컨텍스트, 최소한의 앱 실행을 전제로 준비해 주세요.

참고 문서

- LM Studio MLX Engine Blog
- Apple MLX GitHub
- Apple MLX LM GitHub
- MLX LoRA / QLoRA Guide
- Hugging Face MLX Community

문서 09

docs/09-gemma4-benchmarks-and-agent-expectations.md

Gemma 4 벤치마크와 코딩 에이전트 기대치

왜 이 문서가 필요한가요?

Gemma 4는 로컬에서 실행 가능한 매우 강력한 오픈 모델이지만, 모델 크기에 따라 체감 성능 차이가 큽니다. 특히 E2B, E4B는 작은 노트북에서도 돌아가도록 설계된 모델이라, Codex, Claude Code 같은 상용 코딩 에이전트 수준의 성능을 그대로 기대하면 차이가 크게 느껴질 수 있습니다.

이 문서는 공식 Gemma 4 모델 카드 기준 벤치마크를 정리하고, 실습에서 어떤 기대치를 가져야 하는지 설명합니다.

공식 벤치마크 요약

아래 수치는 Google 공식 Gemma 4 모델 카드에 나온 수치입니다.

모델	MMLU Pro	LiveCodeBench v6	Codeforces ELO	Tau2	MRCR v2 8 needle 128k
Gemma 4 E2B	60.0	44.0	633	24.5	19.1
Gemma 4 E4B	69.4	52.0	940	42.2	25.4
Gemma 4 26B A4B	82.6	77.1	1718	68.2	44.1
Gemma 4 31B	85.2	80.0	2150	76.9	66.4

읽는 법:

- `LiveCodeBench v6`, `Codeforces ELO`는 코딩 성능을 볼 때 참고하기 좋습니다.
- `Tau2`는 에이전트형 작업을 볼 때 참고하기 좋은 지표입니다.
- `MRCR v2 8 needle 128k`는 긴 문맥 처리 능력을 볼 때 참고할 수 있습니다.

모델별 해석

E2B

- 가장 작은 경량 모델입니다.
- 로컬 데모, 간단한 질의응답, 가벼운 보조 작업에는 적합합니다.
- 하지만 코딩과 에이전트 성능은 상위 모델과 차이가 큽니다.
- `LiveCodeBench 44.0`, `Tau2 24.5` 수준이라, 복잡한 코드 수정, 다단계 계획, 도구 사용이 많은 작업에서는 한계가 빨리 드러날 수 있습니다.

권장 사용:

- 8GB 환경
- 최소 사양 테스트
- "로컬 LLM이 돌아간다"는 확인용

E4B

- 실습 기본 모델로 가장 현실적인 선택지입니다.

- E2B보다 확실히 낫지만, 여전히 소형 모델입니다.
- `LiveCodeBench 52.0`, `Tau2 42.2`로 E2B보다 개선되지만, 26B/31B와는 차이가 큽니다.

권장 사용:

- 16GB 전후 노트북
- 일반적인 질의응답, 문서 정리, 간단한 코드 보조
- 행사 표준 기본 모델

주의:

- Codex, Claude Code 같은 강한 코딩 에이전트를 써봤던 참가자라면 생각보다 답이 약하거나, 툴 사용이 덜 안정적으로 느껴질 수 있습니다.

26B A4B

- 성능과 속도의 균형이 좋은 상위 옵션입니다.
- A4B는 답을 만들 때 실제로 계산에 참여하는 부분이 약 4B 규모라는 뜻입니다. 공식 모델 카드 기준으로는 총 파라미터가 `25.2B`, 실제 계산에 참여하는 파라미터가 `3.8B`입니다.
- 이 모델은 여러 "전문가" 부분 중 필요한 일부만 골라 쓰는 `MoE` 방식이라서 계산량은 줄어듭니다. 다만 모델 파일은 전체 26B 규모를 메모리에 올려야 합니다.
- 공식 모델 카드도 이 모델을 31B 대비 빠른 로컬 추론에 유리한 선택지로 설명합니다.
- `LiveCodeBench 77.1`, `Tau2 68.2`로 E4B보다 큰 폭으로 올라갑니다.

권장 사용:

- 32GB 이상
- 코딩, 계획, 에이전트형 흐름을 좀 더 안정적으로 쓰고 싶은 경우
- 31B보다 속도를 더 중시할 때

31B

- Gemma 4 계열 중 가장 높은 일반 성능을 보여주는 상위 모델입니다.
- `LiveCodeBench 80.0`, `Codeforces 2150`, `Tau2 76.9`로 공식 벤치마크에서 가장 강합니다.
- 하드웨어가 허용하면 가장 좋은 선택입니다.

권장 사용:

- 36GB 이상
- 긴 문맥, 코딩, 복잡한 작업, 더 높은 응답 품질이 중요할 때

코딩 에이전트 사용 시 기대치

공식 Gemma 4 모델 카드는 Gemma 4가 함수/도구 호출(function calling) 을 지원하고, reasoning, coding, agentic workflow에 적합하다고 설명합니다. 하지만 모델 크기별 벤치마크 차이를 보면, 특히 E2B, E4B는 상위 모델과 성능 차이가 큽니다.

- E2B, E4B는 `LiveCodeBench`, `Codeforces`, `Tau2`, `MRCR` 같은 코딩·에이전트 프록시 지표에서 26B/31B보다 낮습니다.

- 따라서 작은 모델일수록 복잡한 코드 수정, 긴 지시 처리, 다단계 계획에서 한계가 더 빨리 드러날 가능성이 높습니다.
- Gemma 4는 로컬 실습과 개인 환경에서 매우 유용합니다.
- 하지만 E2B, E4B에 대해 Codex, Claude Code 수준의 코딩 에이전트 경험을 그대로 기대하면 안 됩니다.
- 툴 사용 흐름의 안정성은 공식 벤치마크가 직접 측정한 항목이 아니라, 실행 도구, 프롬프트 구성, 로컬 OpenAI 호환 서버에 따라 달라질 수 있습니다.
- 작은 모델에서는 코드 수정 정확도, 긴 작업 지시 처리, 다단계 계획, 툴 사용 일관성이 약하게 느껴질 수 있습니다.

이번 핸즈온에서는 "상용 코딩 에이전트 대체"보다 로컬에서 실행 가능한 Gemma 4 체험과 기본 워크플로우 이해에 초점을 맞추면 됩니다.

모델별 권장 기대치

- 8GB: E2B는 "최소 실행 보장"에 가깝습니다.
- 16GB: E4B는 가장 현실적인 기본 모델입니다.
- 32GB 이상: 코딩/에이전트 체감을 더 원하면 26B A4B를 권장합니다.
- 36GB 이상: 가능하면 31B가 가장 좋습니다.
- 모델별 구조 차이를 더 자세히 보려면 Gemma 4 아키텍처 상세 정리 문서를 확인해 주세요.

OpenCode 같은 코딩 에이전트에 연결할 때

- opencode 자체는 로컬 모델과 LM Studio를 공식적으로 지원하므로, 개발자 에이전트 시연 용도로는 조건부로 적절합니다.
- 다만 OpenCode 공식 문서도 코드 생성과 도구 호출(tool calling)을 모두 잘하는 모델은 많지 않다고 설명합니다.
- 따라서 Gemma 4를 opencode에 붙일 때는:
- E2B: 비추천
- E4B: 가벼운 보조 수준
- 26B A4B, 31B: 더 현실적인 선택
- 자세한 설치와 연동 방법은 OpenCode 설치 및 LM Studio 연동 가이드 문서를 확인해 주세요.

공식 참고 링크

- Google Gemma 4 official model card (31B)
- Google Gemma 4 official model card (26B A4B)
- Google Gemma 4 official model card (E2B)
- Ollama Gemma 4 library page

문서 10

docs/10-opencode-lmstudio-developer-agent.md

OpenCode 설치 및 LM Studio 연동 가이드

이 문서는 누구를 위한 문서인가요?

- Gemma 4를 개발자 에이전트로 활용해 보고 싶은 참가자
- PM이 만든 PRD를 바탕으로 코드 구현 흐름을 실습해 보고 싶은 참가자
- LM Studio에 내려받은 로컬 Gemma 4 모델을 opencode와 연결하고 싶은 참가자

먼저 결론

- `opencode + LM Studio + Gemma 4` 조합은 조건부로 적절합니다.
- 이유는 opencode가 공식적으로 로컬 모델 + LM Studio의 OpenAI 호환 로컬 API 주소 구성을 지원하고, 에이전트별 프롬프트 / 모델 / 권한 / 단계 수를 설정할 수 있기 때문입니다.
- 다만 이 조합은 모든 참가자의 기본 실습 경로가 아니라, 개발자 에이전트를 시연하거나 실습해 보고 싶은 참가자를 위한 고급/선택형 경로입니다.

왜 이렇게 보나요?

- OpenCode 공식 문서는 로컬 모델과 LM Studio 구성을 지원합니다.
- 하지만 OpenCode 공식 문서도 코드 생성 + 툴 콜링을 모두 잘하는 모델은 많지 않다고 설명합니다.
- 실제로는 `Gemma 4 E2B/E4B` 같은 소형 모델에서 코딩 에이전트 체감이 약하거나, 툴 사용 흐름이 흔들릴 수 있습니다.
- 커뮤니티 이슈에서도 Gemma 4와 로컬 OpenAI 호환 서버 조합에서 도구 호출(tool calling) 실패나 루프 현상이 보고되어 있어, 100% 안정성을 전제로 한 데모로 잡는 것은 권장하지 않습니다.

권장:

- 발표자 데모: `26B A4B` 또는 `31B` 권장
- 32GB 이상 참가자: 선택형으로 권장 가능
- 16GB 참가자: `E4B`로 가벼운 코딩 보조 정도만 권장
- 8GB 참가자: opencode 기반 개발자 에이전트 실습은 비권장

어떤 방식이 가장 안정적인가요?

PRD를 한 번에 길게 넣고 곧바로 전체 구현을 시키기보다, 아래처럼 2단계로 나누면 더 안정적입니다.

1 PM/Plan 단계

- PRD를 읽고 요구사항, 범위, 제약, 구현 순서를 짧은 체크리스트로 정리

1 Developer 단계

- 정리된 체크리스트를 바탕으로 코드 탐색, 수정, 테스트 수행

이 방식이 좋은 이유:

- 작은 모델일수록 긴 문서와 다단계 작업을 한 번에 처리할 때 흔들리기 쉽습니다.

- 먼저 범위를 좁히면 도구 호출(tool calling)과 코드 수정 정확도가 상대적으로 나아질 수 있습니다.
- 발표 흐름도 더 명확합니다.

즉, 이번 핸즈온에서는

- PM 역할: PRD 정리와 구현 체크리스트 작성
- Developer 역할: 체크리스트 기반 구현

처럼 나누는 구성이 더 적절합니다.

모델 선택 권장

opencode 같은 코딩 에이전트 흐름에서는 일반 채팅보다 모델 요구치가 더 높습니다.

환경	권장 모델	안내
8GB	비권장	opencode 기반 개발자 에이전트 실습은 권장하지 않습니다.
16GB	E4B	코드 읽기, 간단한 수정, 초안 생성 정도만 권장합니다.
32GB	26B A4B 우선	가장 현실적인 로컬 개발자 에이전트 실습 구간입니다.
36GB+	31B 또는 26B A4B	품질 우선이면 31B, 반응성 우선이면 26B A4B를 고려하세요.

짧게 정리하면:

- E2B: 개발자 에이전트용으로는 비추천
- E4B: 가벼운 보조 수준
- 26B A4B: 가장 현실적인 선택
- 31B: 가능하면 가장 좋은 선택

OpenCode 설치

이 가이드는 CLI/TUI 기준으로 설명합니다. OpenCode Desktop 앱도 있지만, 개발자 에이전트 실습은 보통 터미널 기반 흐름이 더 단순합니다.

macOS / Linux

가장 간단한 방법:

```bash
brew install anomalyco/tap/opencode
```

또는 설치 스크립트:

```bash
curl -fsSL https://opencode.ai/install | bash
```

설치 확인:

```bash
opencode --version
```

Windows

OpenCode 공식 문서는 Windows에서는 WSL 사용을 권장합니다.

즉, 이 가이드의 OpenCode 설치와 실행 예시는 PowerShell/CMD가 아니라 WSL 터미널 기준입니다.

권장 순서:

1. PowerShell 관리자 권한으로 WSL 설치

```powershell
wsl --install -d Ubuntu
```

설치가 멈추면:

```powershell
wsl --install --web-download -d Ubuntu
```

1. PowerShell에서 Ubuntu 실행

```powershell
wsl -d Ubuntu
```

1. WSL 터미널에서 OpenCode 설치

```bash
curl -fsSL https://opencode.ai/install | bash
```

설치 확인:

```bash
opencode --version
```

WSL 사용 시 프로젝트가 Windows 파일에 있다면 예시는 아래와 같습니다.

```bash
cd /mnt/c/Users/YourName/project
opencode
```

LM Studio와 연결하기

1. LM Studio에서 Gemma 4 모델 준비

먼저 LM Studio에서 아래 중 하나를 다운로드하고 테스트합니다.

- `google/gemma-4-e4b` 계열의 instruction-tuned / chat-ready 모델
- `google/gemma-4-26b-a4b` 계열의 instruction-tuned / chat-ready 모델
- `google/gemma-4-31b` 계열의 instruction-tuned / chat-ready 모델

가능하면 행사 전 아래까지 확인해 두세요.

- 모델 다운로드 완료
- Chat 탭에서 1회 답변 생성 성공
- Hugging Face나 `llama.cpp` 예시에서는 보통 `...-it` 접미사가 보이지만, LM Studio에서는 모델 ID가 더 짧게 정리되어 보일 수 있습니다.

2. LM Studio 로컬 서버 시작

방법 1. GUI

- LM Studio `Developer` 탭으로 이동
- `Start server`를 켭니다

방법 2. CLI

macOS / Linux / WSL:

```bash
lms server start
```

Windows에서 WSL을 쓰지 않고 LM Studio CLI만 확인할 때는 PowerShell에서 아래처럼 실행할 수 있습니다.

```powershell
lms server start
```

기본 엔드포인트는 아래입니다.

```text
http://127.0.0.1:1234/v1
```

3. 모델 목록 확인

아래 명령으로 LM Studio 서버가 보는 모델 목록을 확인합니다.

macOS / Linux / WSL:

```bash
curl http://127.0.0.1:1234/v1/models
```

Windows PowerShell:

```powershell
Invoke-RestMethod http://127.0.0.1:1234/v1/models
```

여기서 보이는 실제 모델 identifier 를 확인해 두는 것이 안전합니다. 문서 예시에서는 `google/gemma-4-e4b` 같은 ID를 사용하지만, 실제 환경에서는 내려받은 모델 항목에 따라 표시 문자열이 다를 수 있습니다. 중요한 것은 이름 자체보다 instruction-tuned / chat-ready 모델을 선택했는지 입니다.

`opencode.json` 예시

프로젝트 루트에 `opencode.json` 파일을 두는 방식을 권장합니다.

```json
{
  "$schema": "https://opencode.ai/config.json",
  "model": "lmstudio/google/gemma-4-e4b",
  "provider": {
    "lmstudio": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "LM Studio (local)",
      "options": {
        "baseURL": "http://127.0.0.1:1234/v1"
      },
      "models": {
        "google/gemma-4-e4b": {
          "name": "Gemma 4 E4B (local)"
        },
        "google/gemma-4-26b-a4b": {
          "name": "Gemma 4 26B A4B (local)"
        },
        "google/gemma-4-31b": {
          "name": "Gemma 4 31B (local)"
        }
      }
    }
  },
  "permission": {
    "bash": {
      "*": "ask",
      "git *": "allow",
      "npm *": "allow",
      "pnpm *": "allow",
      "bun *": "allow"
    },
    "edit": "ask"
  }
}
```

중요:

- `model` 값 형식은 `provider/model-id` 입니다.
- 위 예시에서 provider는 `lmstudio`, model-id는 `google/gemma-4-e4b` 입니다.
- 실제 모델 ID가 다르면 `curl http://127.0.0.1:1234/v1/models` 또는 PowerShell의 `Invoke-RestMethod http://127.0.0.1:1234/v1/models` 결과에 맞춰 수정하세요.
- Hugging Face나 `llama.cpp` 예시에서는 `gemma-4-26b-a4b-it` 같은 ID가 더 직접적으로 보일 수 있습니다.

개발자 에이전트 페르소나 예시

OpenCode는 에이전트별로 프롬프트와 권한을 설정할 수 있습니다. 즉, `PM`, `Developer`, `Reviewer` 같은 역할을 분리하는 구성이 가능합니다.

행사에서는 다음처럼 단순하게 구성합니다.

- `plan` 또는 PM 역할: PRD 해석, 범위 정리

- `developer` 역할: 코드 구현

예시 `opencode.json` 확장:

```json
{
  "$schema": "https://opencode.ai/config.json",
  "model": "lmstudio/google/gemma-4-e4b",
  "provider": {
    "lmstudio": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "LM Studio (local)",
      "options": {
        "baseURL": "http://127.0.0.1:1234/v1"
      },
      "models": {
        "google/gemma-4-e4b": {
          "name": "Gemma 4 E4B (local)"
        },
        "google/gemma-4-26b-a4b": {
          "name": "Gemma 4 26B A4B (local)"
        }
      }
    }
  },
  "agent": {
    "developer": {
      "mode": "primary",
      "model": "lmstudio/google/gemma-4-26b-a4b",
      "prompt": "{file:./prompts/developer-agent.txt}",
      "steps": 6,
      "permission": {
        "bash": {
          "*": "ask",
          "git *": "allow",
          "npm *": "allow",
          "pnpm *": "allow",
          "bun *": "allow"
        },
        "edit": "ask"
      }
    }
  }
}
```

권장 모델 치환:

- 16GB 전후: `developer.model`도 `lmstudio/google/gemma-4-e4b`로 두세요.
- 32GB 이상: `lmstudio/google/gemma-4-26b-a4b` 권장
- 36GB 이상: 필요하면 `lmstudio/google/gemma-4-31b` 고려

예시 `prompts/developer-agent.txt`:

```text
You are the Developer employee in this AI Office.

Your input is a PRD or a short implementation checklist created by the PM.

Your job:
1. Summarize the task briefly.
2. Inspect the codebase before changing anything.
3. Propose a small implementation plan.
4. Make the smallest reasonable code change.
5. Report assumptions, risks, and test results clearly.

Rules:
- Do not make unrelated changes.
- Prefer small diffs.
- If the PRD is too broad, ask to narrow the scope.
- If tool use fails or becomes unstable, fall back to a text plan instead of hallucinating
success.
```

실제 사용 순서 예시

1. 프로젝트 폴더에서 `opencode` 실행
2. 필요하면 `/models` 로 `lmstudio/google/gemma-4-e4b` 또는 `lmstudio/google/gemma-4-26b-a4b` 선택
3. `plan` 또는 PM 역할로 PRD를 짧은 구현 체크리스트로 정리
4. `developer` 에이전트로 전환
5. 체크리스트를 넣고 구현 요청

예시 흐름:

```text
이 PRD를 읽고 이번 세션에서 구현할 최소 범위를 5개 이하 체크리스트로 정리해 줘.
```

그 다음:

```text
이 체크리스트를 기준으로 현재 코드베이스를 확인하고, 가장 작은 단위부터 구현해 줘.
```

OpenCode 실습 주의사항

- `opencode`는 이번 행사에서 선택형 고급 경로입니다.
- E2B, E4B는 일반 채팅보다 코딩 에이전트 체감이 약할 수 있습니다.
- 특히 작은 로컬 모델은 도구 호출(tool calling), 긴 PRD 처리, 다단계 수정 흐름에서 흔들릴 수 있습니다.
- 짧은 PRD, 좁은 구현 범위, 작은 단계, `ask` 기반 권한 설정을 권장합니다.
- 가능하면 26B A4B 또는 31B 모델에서 실습하는 것이 좋습니다.

참고 링크

- OpenCode Intro

- OpenCode Providers
- OpenCode Models
- OpenCode Agents
- OpenCode Permissions
- OpenCode Windows (WSL)
- OpenCode GitHub
- LM Studio local server
- LM Studio OpenAI 호환 endpoint 문서
- LM Studio List Models
- OpenCode issue: Gemma 4 e4b tool calling fails via Ollama
- OpenCode issue: Gemma 4 26B / 31B interaction issues

더 가벼운 대안은 Pi 경량 코딩 에이전트 대안과 이번 핸즈온 선택 이유 문서를 확인해 주세요.

문서 11

docs/11-pi-and-tool-selection-notes.md

Pi 경량 코딩 에이전트 대안과 이번 핸즈온 선택 이유

이 문서는 누구를 위한 문서인가요?

- `opencode`보다 더 가벼운 터미널 코딩 에이전트를 찾는 참가자
- Gemma 4 local LLM을 붙일 수 있는 다른 코딩 에이전트 경로를 알고 싶은 참가자
- 왜 이번 핸즈온에서 `gemini-cli`, `Antigravity`를 code agent로 사용하지 않았는지 궁금한 참가자

먼저 결론

- `pi`는 좋은 경량 대안입니다.
- 이유는 `pi`가 스스로를 minimal terminal coding harness 로 설명하고, `~/.pi/agent/models.json` 을 통해 LM Studio / Ollama / vLLM 같은 로컬 OpenAI 호환 서버를 붙일 수 있다고 공식 문서에 적고 있기 때문입니다.
- 이번 핸즈온은 Gemma 4를 참가자 노트북에서 미리 내려받아 두고, 네트워크가 없거나 매우 불안정해도 로컬에서 실행 가능한 흐름을 중요하게 봅니다.
- 반대로 `gemini-cli`와 `Antigravity`는 이번 핸즈온 기준 Gemma 4 local LLM + LM Studio 기반의 오프라인 code agent 경로가 명확하지 않습니다.

짧게 정리하면:

- `opencode`: 기능이 더 많고, 역할 분리와 에이전트 구성이 쉬움
- `pi`: 더 가볍고 단순한 경량 대안
- `gemini-cli`: 공식 문서 기준 Gemini 중심 도구
- `Antigravity`: 공식 공개 자료 기준 hosted model 중심 도구에 가까우며, 로컬 Gemma 4 + LM Studio 기반 오프라인 경로가 명확하지 않음

`pi`는 왜 대안으로 적절한가요?

Pi 공식 README 기준:

- `pi`는 minimal terminal coding harness 입니다.
- 기본 도구는 `read`, `write`, `edit`, `bash` 로 비교적 단순합니다.
- `sub agents`와 `plan mode` 같은 기능은 기본 내장보다 확장/패키지 방식에 가깝습니다.
- `~/.pi/agent/models.json` 으로 custom providers & models 를 추가할 수 있고, 문서에서 Ollama, LM Studio, vLLM 같은 로컬 모델 서버를 명시적으로 언급합니다.

즉, 이번 핸즈온에서는:

- `opencode`보다 설정 면적이 작고
- 로컬 OpenAI 호환 서버 연결이 가능하며
- "개발자 에이전트 최소 버전"을 보여주기 좋습니다.

`opencode`와 `pi`는 어떻게 나눠 쓰나요?

도구	추천 상황	특징
`opencode`	역할 분리, 에이전트 구성, 데모 확장성이 중요할 때	기능이 더 많고, 에이전트/권한/프롬프트 구성이 풍부함
`pi`	더 가볍고 단순한 CLI 코딩 에이전트를 원할 때	최소 도구 구성, 빠른 시작, 커스텀 모델 연결 가능

권장 사용:

- 발표자 데모 메인 경로: `opencode`
- 가벼운 대안 소개: `pi`

`pi` 설치 가이드

공식 Quick Start 기준:

macOS / Linux:

```bash
npm install -g @mariozechner/pi-coding-agent
```

Windows PowerShell:

```powershell
npm install -g @mariozechner/pi-coding-agent
```

설치 확인:

macOS / Linux:

```bash
pi --version
```

Windows PowerShell:

```powershell
pi --version
```

Windows 메모:

- `pi` 자체는 PowerShell에서도 설치와 실행이 가능합니다.
- 다만 이번 핸즈온의 code agent 메인 경로는 `OpenCode`이고, OpenCode는 Windows에서 WSL 권장입니다.
- Windows에서는 pi = PowerShell 가능, OpenCode = WSL 권장으로 구분하면 됩니다.

Ollama를 함께 쓰는 참가자는 `ollama launch pi --config`로 Pi의 Ollama provider 설정을 자동 생성할 수도 있습니다. 이 문서에서는 LM Studio 연결을 직접 확인하기 위해 `models.json` 수동 설정을 기준으로 설명합니다.

LM Studio와 `pi` 연결하기

1. LM Studio 서버 준비

먼저 LM Studio에서 Gemma 4 모델을 다운로드하고, `Developer` 탭에서 서버를 시작합니다.

핸즈온 기준으로는 instruction-tuned / chat-ready 모델을 선택합니다. Hugging Face나 `llama.cpp` 예시에서는 보통 `...-it` 접미사가 보이지만, LM Studio에서는 이름이 더 짧게 보일 수 있습니다.

기본 엔드포인트:

```text
http://127.0.0.1:1234/v1
```

모델 확인:

macOS / Linux:

```bash
curl http://127.0.0.1:1234/v1/models
```

Windows PowerShell:

```powershell
Invoke-RestMethod http://127.0.0.1:1234/v1/models
```

2. `~/.pi/agent/models.json` 설정

Pi 공식 문서 기준으로, local model provider는 `models.json` 으로 추가합니다.

Pi 모델 문서 기준으로 `openai-completions` 가 most compatible 한 기본 선택입니다.

예시:

```json
{
  "providers": {
    "lmstudio": {
      "baseUrl": "http://127.0.0.1:1234/v1",
      "api": "openai-completions",
      "authHeader": false,
      "apiKey": "lm-studio",
      "models": [
        { "id": "google/gemma-4-e4b", "name": "Gemma 4 E4B (LM Studio)" },
        { "id": "google/gemma-4-26b-a4b", "name": "Gemma 4 26B A4B (LM Studio)" },
        { "id": "google/gemma-4-31b", "name": "Gemma 4 31B (LM Studio)" }
      ]
    }
  }
}
```

중요:

- 여기서 `id` 는 LM Studio 서버가 실제로 노출하는 모델 ID와 맞아야 합니다.
- 먼저 `curl http://127.0.0.1:1234/v1/models` 또는 PowerShell의 `Invoke-RestMethod http://127.0.0.1:1234/v1/models` 로 확인한 뒤 맞추세요.
- 예시처럼 `authHeader: false` 로 두면 LM Studio에 불필요한 Bearer 인증 헤더를 보내지 않게 됩니다.

- 만약 로컬 서버가 OpenAI 호환 필드를 일부만 지원한다면, Pi 모델 문서의 `compat` 설정으로 추가 조정할 수 있습니다.

3. 실행

실행 후 `/model` 로 모델을 고르거나, provider/model 형식으로 선택합니다.

예시:

macOS / Linux:

```bash
pi --model lmstudio/google/gemma-4-e4b
```

Windows PowerShell:

```powershell
pi --model lmstudio/google/gemma-4-e4b
```

또는 interactive 모드에서:

```text
/model
```

`pi`에서 Gemma 4를 쓸 때 권장안

- 8GB: 비권장
- 16GB: E4B로 가벼운 코드 보조만 권장
- 32GB: 26B A4B 권장
- 36GB+: 31B 또는 26B A4B

`pi`도 결국 코딩 에이전트이므로, 작은 모델의 한계는 `opencode`와 비슷하게 존재합니다.

왜 `gemini-cli`를 기본 code agent로 쓰지 않나요?

`gemini-cli`는 좋은 터미널 AI 에이전트이지만, 이번 핸즈온의 기본 code agent로는 쓰지 않습니다.

이번 핸즈온의 기준은 아래입니다.

- 참가자 노트북에서 Gemma 4를 미리 다운로드
- LM Studio, Ollama, llama.cpp 같은 로컬 실행 경로 사용
- 현장 인터넷이 불안정해도 기본 실습 진행 가능

`gemini-cli`는 공식 문서상 Gemini, Google 계정, Gemini API key, Vertex AI 중심의 사용 흐름이 먼저 나옵니다. 로컬 Gemma 4를 LM Studio 또는 Ollama의 OpenAI 호환 로컬 API 주소로 붙이는 경로는 이번 사전 준비의 기본 경로로 잡기 어렵습니다.

따라서 이번 핸즈온에서는:

- 기본 고급 code agent: `OpenCode`
- 더 가벼운 대안: `pi`

- Gemini 중심 워크플로: 별도 실험 경로

로 구분합니다.

왜 `Antigravity`를 기본 code agent로 쓰지 않나요?

`Antigravity`도 이번 핸즈온의 기본 code agent로는 쓰지 않습니다.

이유는 단순합니다.

- 공개 공식 자료는 personal Gmail account와 preview 사용 흐름을 전제로 설명합니다.
- 설치, 에이전트 사용, rules/workflows, 보안 설정은 확인할 수 있지만, LM Studio나 localhost의 OpenAI 호환 로컬 API 주소에 로컬 Gemma 4를 연결하는 안정적인 기본 경로는 확인하기 어렵습니다.
- 따라서 이번 핸즈온의 핵심 조건인 로컬 Gemma 4 + 사전 다운로드 + 오프라인 가능 실습과는 맞지 않습니다.

Antigravity는 hosted model 중심 agent IDE를 써 보고 싶은 별도 실험 주제로 보면 됩니다. 이번 사전 준비에서는 로컬 실행이 쉬운 `OpenCode`, 그리고 더 가벼운 `pi`를 우선합니다.

참고 링크

- Pi coding agent README
- Pi custom models docs
- Ollama Pi integration
- OpenCode 설치 및 LM Studio 연동 가이드
- Gemini CLI README
- Gemini CLI `/model` docs
- Gemini CLI issue: local/offline models
- Gemini CLI discussion: OpenAI-compatible local model support
- Google Antigravity Codelab

문서 12

docs/12-hermes-agent-overview.md

Hermes Agent란 무엇인가

이 문서는 누구를 위한 문서인가요?

- `Hermes Agent`가 정확히 어떤 도구인지 먼저 이해하고 싶은 참가자
- `Codex`, `Claude Code`, `OpenCode` 같은 코딩 에이전트와 무엇이 다른지 감을 잡고 싶은 참가자
- 왜 이번 핸즈온에서 `Hermes`를 기본 경로가 아니라 선택형 고급 경로로 두는지 알고 싶은 참가자

먼저 한 줄 정의

`Hermes Agent`는 Nous Research가 만든 자율형 AI agent runtime 입니다.

단순 채팅 앱이나 IDE 안에 묶인 코딩 보조 도구라기보다:

- 도구를 호출하고
- 기억을 쌓고
- skills를 재사용하고
- 여러 플랫폼에서 동작하고
- 필요하면 subagent로 작업을 나누는

장기 실행형 agent framework 에 가깝습니다.

공식 문서가 설명하는 Hermes의 핵심

Hermes 공식 문서 기준으로 핵심 포인트는 아래입니다.

- self-improving agent
- built-in learning loop
- persistent memory
- skills system
- context files
- scheduled tasks
- subagent delegation
- messaging gateway
- provider-agnostic model routing

즉, 모델 하나에 얇게 붙는 채팅 UI가 아니라, 기억 + 도구 + 자동화 + 실행 환경까지 포함한 agent 시스템으로 이해하면 됩니다.

Hermes는 무엇을 할 수 있나요?

Hermes Features Overview 기준으로 주요 기능은 아래와 같습니다.

1. Tools / Toolsets

- 터미널 실행

- 파일 읽기/쓰기
- 웹 검색/추출
- 브라우저 자동화
- 이미지/음성 관련 도구

즉, 모델이 답만 하는 것이 아니라 실제로 작업을 수행할 수 있습니다.

2. Skills System

- 필요한 지식을 on-demand로 로드
- 토큰 낭비를 줄이기 위한 progressive disclosure
- 재사용 가능한 플레이북 축적

핸즈온 관점에서는 이 부분이 특히 중요합니다.

3. Persistent Memory

- `MEMORY.md`
- `USER.md`

같은 형태로 세션 간 기억을 이어 갑니다.

4. Context Files

- `.hermes.md`
- `AGENTS.md`
- `CLAUDE.md`
- `SOUL.md`

같은 파일을 읽어 프로젝트 규칙과 정체성을 반영합니다.

5. Delegation / Automation

- `delegate_task`로 child agent 생성
- `cron` 기반 자동화
- 여러 플랫폼으로 결과 전달

즉 Hermes는 “한 번 답하는 도구”보다 “계속 일하는 agent”에 더 가깝습니다.

다른 도구와 무엇이 다른가요?

짧게 비교하면:

- `LM Studio`: 로컬 모델 실행/관리 GUI
- `Ollama`: 로컬 모델 실행 CLI/API
- `llama.cpp`: GGUF 실행 엔진 및 로컬 서버
- `OpenCode`: 코딩 중심 에이전트
- `Hermes`: 코딩도 할 수 있지만, 더 넓게는 에이전트 운영체제에 가까움

즉:

- `LM Studio`, `Ollama`, `llama.cpp`는 주로 추론 서버
- `Hermes`는 그 위에 붙는 agent layer

라고 보면 됩니다.

이번 핸즈온에서 Hermes를 어떻게 보나요?

이번 핸즈온에서는 `Hermes`를 선택형 고급 경로로 봅니다.

이유:

- Windows에서는 WSL2가 필요합니다.
- 설치와 설정 면적이 기본 경로보다 넓습니다.
- 모든 참가자가 필수로 준비하기에는 설치와 연결 과정이 복잡합니다.

반대로 강점도 분명합니다.

- role-based agent workflow를 구성하기 좋음
- skills 실습 주제로 적합함
- 로컬 OpenAI 호환 서버와 연결 가능
- `Gemma 4 + llama.cpp` 또는 `Gemma 4 + LM Studio` 위에 agent layer를 얹을 수 있음

이번 핸즈온에서 Hermes를 어떤 그림으로 이해하면 되나요?

가장 쉬운 그림은 이렇습니다.

```text
Gemma 4
  -> local inference server
     (LM Studio / Ollama / llama.cpp)
        -> Hermes Agent
           -> tools, skills, memory, delegation
```

즉 `Gemma 4`는 두뇌이고, `Hermes`는 실행기이자 오케스트레이터입니다.

왜 기본 경로가 아니고 고급 경로인가요?

- 기본 참가자는 `LM Studio`만으로도 충분히 세션을 따라올 수 있습니다.
- `Hermes`는 설치뿐 아니라 모델 endpoint 연결까지 이해해야 합니다.
- Windows 참가자는 WSL2까지 준비해야 합니다.
- 핸즈온 60분 안에 모두에게 같은 수준의 안정성을 기대하기 어렵습니다.

그래서 문서 흐름상으로도:

1. 모델과 실행 환경 이해
2. 기본 설치 경로 준비
3. 그 다음 고급 경로로 `Hermes` 확장

순서가 자연스럽습니다.

누구에게 특히 잘 맞나요?

- 에이전트 워크플로에 관심 있는 참가자
- `skills`를 직접 만들어 보고 싶은 참가자
- role-based workflow를 구성해 보고 싶은 참가자
- 로컬 모델 위에 agent system을 얹어 보고 싶은 참가자

다음에 무엇을 보면 되나요?

- 실제 설치는 Hermes Agent 설치 가이드
- 로컬 서버는 llama.cpp 설치 가이드
- 코드 에이전트 기대치는 Gemma 4 벤치마크와 코딩 에이전트 기대치

공식 참고 링크

- Hermes Docs Home
- Features Overview
- Quickstart

문서 13

docs/13-hermes-agent-setup.md

Hermes Agent 설치 가이드

Hermes가 어떤 도구인지 먼저 이해하고 싶다면 Hermes Agent란 무엇인가 문서를 먼저 확인해 주세요.

이 문서는 누구를 위한 문서인가요?

- Hermes Agent를 이번 핸즈온의 선택형 고급 경로로 써 보고 싶은 참가자
- Gemma 4 + llama.cpp 또는 다른 OpenAI 호환 로컬 서버에 Hermes를 연결해 보고 싶은 참가자
- CEO / CPO / PM / QA / Developer 같은 역할 기반 AI office 데모를 직접 구성해 보고 싶은 참가자

먼저 결론

- Hermes Agent는 이번 세션의 고급 / 선택형 경로입니다.
- macOS, Linux, WSL2에서는 설치가 비교적 단순합니다.
- Windows 네이티브는 공식 미지원이며, Windows 사용자는 WSL2 안에서 설치해야 합니다.
- 발표자 데모나 고급 참가자 실습에는 좋지만, 모든 참가자의 기본 경로로 두기에는 설치와 연결 과정이 복잡합니다.

사전 안내

- 행사 전에 반드시 설치를 끝내고 오세요.
- 행사 전에 최소 1회 hermes 실행 테스트를 해 두세요.
- Hermes만 설치된 상태로는 로컬 모델 추론이 되지 않습니다.
- 로컬 모델을 쓰려면 Ollama, LM Studio, llama.cpp, vLLM 같은 별도 inference server가 필요합니다.

공식 지원 범위

macOS / Linux / WSL2

공식 one-line installer를 사용할 수 있습니다.

```bash
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
```

설치가 끝나면 셸을 다시 읽습니다.

```bash
source ~/.zshrc
```

또는 bash 사용자라면:

```bash
source ~/.bashrc
```

Windows

Windows 네이티브는 공식 미지원입니다. 먼저 `WSL2`를 설치한 뒤, WSL 터미널 안에서 위 설치 명령을 실행해야 합니다.

Windows 사용자는 Hermes = WSL2 전용 선택 경로로 이해하면 됩니다.

PowerShell 관리자 권한에서 먼저 할 일:

```powershell
wsl --install -d Ubuntu
```

설치가 멈추거나 Store 경로가 불안정하면:

```powershell
wsl --install --web-download -d Ubuntu
```

설치 후 상태 확인:

```powershell
wsl --status
wsl --list --verbose
```

Ubuntu 실행:

```powershell
wsl -d Ubuntu
```

설치 전에 확인할 것

- `git --version`
- 관리자 권한 또는 설치 권한
- 안정적인 인터넷
- 여유 디스크 공간
- macOS / Linux / WSL2 셸 사용 가능 여부

`Hermes` 설치 스크립트는 공식 문서 기준으로 아래를 자동 처리합니다.

- `uv`
- Python 3.11
- Node.js
- `ripgrep`
- `ffmpeg`
- `hermes` 명령 연결
- setup wizard 실행

설치 순서

1. Windows 사용자는 먼저 WSL2 준비

PowerShell 관리자 권한:

```powershell
wsl --install -d Ubuntu
```

설치 확인:

```powershell
wsl --status
wsl --list --verbose
```

Ubuntu 셸 열기:

```powershell
wsl -d Ubuntu
```

이후 명령은 WSL 터미널 안에서 실행합니다.

macOS / Linux 사용자는 이 단계를 건너뛰고 바로 다음 단계로 가면 됩니다.

2. 설치 실행

```bash
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
```

3. 셸 재로드

```bash
source ~/.zshrc
```

또는:

```bash
source ~/.bashrc
```

4. 설정 마법사 또는 모델 설정

처음 설치 후 setup wizard가 열리면 기본 모델 공급자를 설정할 수 있습니다.

나중에 다시 바꾸려면:

```bash
hermes model
```

5. 설치 확인

```bash
hermes version
hermes doctor
```

로컬 모델과 연결하는 법

이번 핸즈온에서는 `Hermes`가 직접 모델을 돌리는 것이 아니라, 별도 로컬 서버에 붙는 구조로 이해하면 됩니다.

중요:

- Hermes Agent는 다단계 tool-calling 작업을 위해 최소 64K 토큰 컨텍스트를 요구합니다.
- 로컬 모델 서버의 컨텍스트가 이보다 작으면 Hermes가 시작 단계에서 거부하거나, 작업 중 불안정하게 동작할 수 있습니다.
- 컨텍스트를 키우면 메모리 사용량도 함께 늘어나므로, 8GB/16GB 장비에서는 Hermes 경로를 무리하게 권장하지 않습니다.

도구별 확인 예시:

- `llama.cpp`: `llama-server ... --ctx-size 65536`
- `Ollama`: 필요 시 `OLLAMA_CONTEXT_LENGTH=64000 ollama serve`처럼 64K 이상으로 서버를 띄운 뒤 연결
- `LM Studio`: 서버를 켜기 전에 모델 설정에서 context length를 64K 이상으로 설정할 수 있는지 확인

따라서 Hermes와 연결할 로컬 서버는 아래 두 가지를 먼저 확인해야 합니다.

1. `/v1/models`에서 실제 model ID가 보이는지
2. 해당 서버/모델이 64K 이상 컨텍스트로 실행 중인지

예를 들어 `llama.cpp`의 `llama-server`를 `127.0.0.1:8080/v1`에 띄웠다면:

```bash
hermes model
```

입력 예시:

- Base URL: `http://127.0.0.1:8080/v1`
- API key: none
- Model name: `/v1/models`에서 보이는 모델 ID
- Context length: `64000` 또는 `65536` 이상

즉 구조는 아래와 같습니다.

```text
Hermes Agent -> OpenAI 호환 로컬 서버 -> Gemma 4
```

Windows 참가자에게 권하는 방식

Windows에서 `Hermes`를 꼭 쓰고 싶다면 아래 방식으로 준비하세요.

1. Windows에서 `LM Studio` 또는 `llama.cpp` 서버 실행
2. WSL2 안에서 `Hermes` 실행
3. `Hermes`가 로컬 서버 주소에 연결

이 방식은 WSL 안에서 대용량 모델을 다시 내려받고 빌드하는 부담을 줄여 줍니다.

Windows PowerShell에서 WSL 진입 예시:

```powershell
wsl -d Ubuntu
```

그 다음 WSL 안에서:

```bash
hermes version
hermes doctor
```

행사 전 최종 확인

아래가 되면 준비 완료입니다.

Hermes 준비 완료 기준

macOS / Linux / WSL2:

```bash
hermes version
hermes doctor
```

그 다음 `hermes` 또는 `hermes --tui`를 실행해 짧은 질문이 응답되는지 확인하세요.

Windows 사용자는 위 명령을 PowerShell이 아니라 WSL2 Ubuntu 셸 안에서 실행하면 됩니다.

로컬 모델 연결까지 확인하려면:

```bash
hermes model
```

에서 로컬 서버 주소를 등록한 뒤, 간단한 질문이 한 번 응답되면 충분합니다.

이 경로를 누구에게 권하나요?

- 발표자 데모
- 고급 참가자
- 로컬 에이전트, skills, delegation, AI office 워크플로를 직접 구성해 보고 싶은 참가자

이 경로를 누구에게는 권하지 않나요?

- 당일 처음 WSL2를 설치해야 하는 Windows 참가자
- 60분 안에 GUI 중심으로만 간단한 체험을 원하는 참가자

- 에이전트보다 기본 로컬 추론만 빨리 체험하고 싶은 참가자

공식 참고 링크

- Hermes Installation
- Hermes Quickstart
- Hermes CLI Commands Reference
- Hermes FAQ
- Hermes Docs Home

문서 14

docs/14-gemma4-architecture-deep-dive.md

Gemma 4 아키텍처 상세 정리

이 문서는 누구를 위한 문서인가요?

- Gemma 4의 구조를 조금 더 깊게 이해하고 싶은 참가자
- E2B, E4B, 26B A4B, 31B가 왜 다른지 알고 싶은 참가자
- 설치 가이드보다 모델 자체의 설계 차이가 궁금한 참가자

먼저 알아둘 점

- 정확한 모델별 수치는 Google 공식 모델 카드 기준으로 정리했습니다.
- Hugging Face의 Gemma 4 blog 와 Maarten Grootendorst의 A Visual Guide to Gemma 4 는 이해를 돕는 설명 자료로 함께 참고했습니다.
- 따라서 아래 문서에서는:
- 모델 크기, 레이어 수, 컨텍스트, modality 같은 항목은 공식 모델 카드 기준
- 왜 이런 구조가 나왔는지에 대한 해설은 블로그/설명 자료를 참고한 해석

으로 이해하면 됩니다.

먼저 용어부터 보기

모델 구조 용어는 세부 구현보다 아래 정도만 이해하면 충분합니다.

- `Dense`: 모델의 주요 계산 블록을 대부분 함께 쓰는 일반적인 구조입니다. 구조가 단순하지만, 큰 모델일수록 계산과 메모리 부담이 커집니다.
- `MoE`: "여러 전문가 중 필요한 일부만 골라 쓰는 구조"입니다. 모델 안에 여러 전문가 블록(expert)이 있고, 답을 만들 때마다 그중 일부만 계산합니다.
- `active parameters`: 답을 만들 때 그 순간 실제 계산에 참여하는 파라미터입니다.
- `effective parameters`: 작은 모델의 체감 계산 규모를 설명하기 위한 표기입니다.
- `PLE`: E2B/E4B가 작은 모델 효율을 높이기 위해 쓰는 보조 장치입니다. 세부 구조보다 "작은 모델을 더 효율적으로 쓰기 위한 장치"로 이해하면 됩니다.

한눈에 보는 구조 차이

모델	구조	핵심 포인트
E2B	Dense + PLE	작은 온디바이스용, audio 지원, 128K context
E4B	Dense + PLE	작은 온디바이스용, audio 지원, 128K context
26B A4B	MoE	총 26B지만 답을 만들 때는 약 4B 규모만 계산, 속도/성능 균형형
31B	Dense	가장 단순하고 강한 상위 모델, 품질 우선

핵심:

- 작은 모델(E2B, E4B)은 PLE 로 효율을 높인 구조입니다.
- 26B A4B는 여러 전문가 중 일부만 골라 쓰는 MoE 구조입니다.
- 31B는 모든 주요 계산 블록을 함께 쓰는 상위 모델입니다.

Gemma 4 전체가 공통으로 갖는 구조

공식 모델 카드와 Hugging Face 기술 블로그 기준으로, Gemma 4 전반에는 아래 같은 공통 특성이 있습니다.

1. Hybrid Attention

Gemma 4는 local sliding-window attention 과 global full attention 을 섞어 씁니다.

- local layer는 계산량과 메모리를 줄이는 역할
- global layer는 긴 문맥과 전체 구조를 다시 보는 역할
- 마지막 레이어는 global attention 이 되도록 설계되었습니다

이 구조 덕분에:

- 긴 컨텍스트를 지원하면서도
- 무조건 모든 레이어가 full attention 인 모델보다
- 더 효율적으로 동작합니다.

2. Long Context용 RoPE 설계

RoPE는 문장 안에서 각 토큰이 어느 위치에 있는지 모델이 이해하도록 돕는 방식입니다.

Hugging Face 설명 자료 기준으로 Gemma 4는:

- sliding layer에는 표준 RoPE
- global layer에는 긴 문맥에 맞춘 Proportional / low-frequency-pruned RoPE

를 사용합니다.

참가자 관점에서는:

- “128K / 256K 문맥을 그냥 억지로 늘린 게 아니라”
- 긴 문맥을 버티기 위한 attention + positional encoding 조합이 들어갔다

정도로 이해하면 충분합니다.

3. Shared KV Cache

KV cache는 모델이 이미 읽은 문맥을 다시 계산하지 않도록 저장해 두는 내부 메모리입니다.

Hugging Face blog 기준으로 Gemma 4는 shared KV cache 를 사용합니다.

- 마지막 일부 레이어가 자기만의 key/value projection을 다시 만들지 않고
- 앞쪽 레이어의 KV 상태를 재사용합니다

의미:

- 긴 문맥에서 메모리와 계산량 절감
- 온디바이스/로컬 추론에 유리

4. Vision Encoder

Gemma 4는 전 모델이 image 입력을 지원합니다.

핵심 특징:

- 가변 aspect ratio
- 가변 resolution
- 고정된 soft token budget 으로 이미지 정보를 요약

Hugging Face blog 기준 대표 token budget:

- `70`
- `140`
- `280`
- `560`
- `1120`

즉, 같은 이미지 모델이라도:

- 더 적은 시각 토큰으로 빠르게 볼 수도 있고
- 더 많은 시각 토큰으로 자세히 볼 수도 있습니다.

5. Audio Encoder

audio 는 E2B / E4B 에만 있습니다.

Hugging Face blog 기준:

- USM-style conformer 기반 audio encoder
- Gemma-3n 계열과 유사한 audio 처리 경로

즉:

- 작은 모델은 text + image + audio
- 큰 모델은 text + image

로 이해하면 됩니다.

모델별 공식 구조 요약

Google 공식 Gemma 4 모델 카드 기준:

Dense 모델

Dense 모델은 답을 만들 때 모델의 주요 계산 블록을 대부분 함께 쓰는 구조입니다.

항목	E2B	E4B	31B
총 파라미터	2.3B effective (5.1B with embeddings)	4.5B effective (8B with embeddings)	30.7B
레이어 수	35	42	60

항목	E2B	E4B	31B
sliding window	512	512	1024
컨텍스트	128K	128K	256K
vocab	262K	262K	262K
modality	Text, Image, Audio	Text, Image, Audio	Text, Image
vision encoder	~150M	~150M	~550M
audio encoder	~300M	~300M	없음

MoE 모델

항목	26B A4B
총 파라미터	25.2B
실제 계산 참여 파라미터	3.8B
레이어 수	30
sliding window	1024
컨텍스트	256K
vocab	262K
전문가 블록 구성	128개 중 8개 선택 + 공통 전문가 블록 1개
modality	Text, Image
vision encoder	~550M

A4B는 무슨 뜻인가요?

26B A4B는 전체 규모는 26B급이지만, 답을 만들 때 실제로 계산에 참여하는 부분은 약 4B 규모인 모델이라는 뜻입니다.

- 26B: 모델 전체가 가진 총 파라미터 규모입니다. 공식 카드 기준 실제 수치는 25.2B입니다.
- A: active, 즉 실제 계산에 참여한다는 뜻입니다.
- 4B: 답을 만들 때 계산에 참여하는 파라미터 규모입니다. 공식 카드 기준 실제 수치는 3.8B입니다.

이 차이는 26B A4B가 MoE(Mixture-of-Experts) 구조이기 때문에 생깁니다. 모델 안에는 여러 전문가 블록이 있고, 답을 만들 때마다 그중 일부만 골라 계산합니다. 공식 카드 기준으로는 전체 128개 전문가 블록 중 8개와 1개 공통 전문가 블록을 사용합니다.

참가자 입장에서 가장 중요한 해석은 아래입니다.

- 속도: 답을 만들 때 약 4B 규모만 계산하므로 31B보다 빠르게 느껴질 수 있습니다.
- 품질: 전체 모델 규모는 26B급이라 E2B/E4B보다 더 좋은 코딩과 에이전트 체감을 기대할 수 있습니다.
- 메모리: 전체 26B 규모의 가중치를 메모리에 올려야 하므로, 4B 모델처럼 가볍게 실행되는 것은 아닙니다.

E2B / E4B는 왜 “effective” 인가요?

공식 모델 카드와 Hugging Face blog 기준으로, 작은 모델에는 Per-Layer Embeddings (PLE) 가 들어갑니다.

이 구조 때문에:

- 총 embedding 테이블 크기는 꽤 크지만
- 실제 추론에서 답을 만들 때 무겁게 계산되는 파라미터는 더 적습니다

그래서 공식 카드도:

- `E2B = 2.3B effective (5.1B with embeddings)`
- `E4B = 4.5B effective (8B with embeddings)`

처럼 표기합니다.

직관적으로는:

- "모델 본체는 더 작게 유지하면서"
- "레이어별 추가 조건 신호를 embedding lookup 방식으로 넣어"
- 온디바이스 효율을 높인 구조

라고 이해하면 됩니다.

Hugging Face blog 해설 기준으로 PLE는:

- 각 token에 대해 레이어마다 작은 보조 벡터를 공급하고
- 각 레이어가 자기 시점에 필요한 token-specific 정보를 더 잘 받게 해줍니다

즉, 작은 모델에서 깊이를 무작정 늘리지 않고 효율을 높이는 장치라고 볼 수 있습니다.

26B A4B는 왜 빠를 수 있나요?

26B A4B는 위에서 설명한 MoE 구조를 사용합니다.

의미:

- 모델 전체는 26B급으로 크지만
- 매 토큰마다 전체를 다 활성화하지 않고
- 일부 전문가 블록만 선택적으로 계산합니다

그래서:

- 31B보다 더 가볍게 느껴질 수 있고
- 로컬 환경에서는 속도/품질 균형이 좋을 수 있습니다

다만 중요한 점:

- 메모리에는 전체 파라미터가 올라가야 하므로
- "4B 모델처럼 가볍게 메모리만 쓰는 것"은 아닙니다

즉:

- 속도 관점에서는 4B active의 장점
- 메모리 관점에서는 26B total의 부담

을 동시에 갖습니다.

31B는 왜 따로 의미가 있나요?

31B는 Gemma 4 계열의 상위 모델입니다.

장점:

- 구조가 비교적 단순하고 해석이 쉬움
- 최고 수준의 일반 성능
- coding / reasoning / long-context 품질이 가장 좋음

공식 카드와 Hugging Face 설명을 같이 보면:

- 31B는 "특별한 트릭"보다
- Gemma 4 공통 구조를 더 큰 규모로 밀어붙인 상위 모델

로 이해하면 됩니다.

이번 핸즈온에서는:

- 가능한 하드웨어가 있으면 가장 품질이 좋음
- 다만 일반 노트북에서는 부담이 큼

Vision 쪽에서 Gemma 4가 중요한 이유

Gemma 4는 text-only 모델이 아니라 multimodal 모델입니다.

특히 image 쪽에서 중요한 점:

- 정사각형으로 무조건 찌그러뜨리지 않고
- 원본 aspect ratio를 더 잘 보존하려고 하고
- token budget을 선택해 속도/메모리/품질을 조절할 수 있습니다

그래서:

- OCR
- 문서 읽기
- UI 이해
- 화면 인식

같은 시나리오에서 활용도가 높습니다.

핸즈온 관점에서 이 구조 차이가 왜 중요할까요?

E2B / E4B

- 로컬 노트북 친화적
- audio 지원
- 하지만 코딩 에이전트나 복잡한 계획에는 한계가 더 빨리 드러날 수 있음

26B A4B

- 로컬에서 코딩/에이전트 체감을 노리기 좋은 균형형

- 속도와 품질의 절충안으로 보기 좋음

31B

- 가장 좋은 품질
- 발표자 데모나 고사양 환경에 적합

한 줄 해석

- E2B/E4B: 작은 온디바이스 모델 + PLE
- 26B A4B: 빠른 상위 MoE
- 31B: 가장 강한 상위 모델

같이 보면 좋은 문서

- 하드웨어 / 운영체제 / 모델 선택 가이드
- 메모리 기준으로 Gemma 4 모델 고르는 방법
- Gemma 4 벤치마크와 코딩 에이전트 기대치

참고 링크

- Google Gemma 4 E2B model card
- Google Gemma 4 26B A4B model card
- Google Gemma 4 31B model card
- Gemma 4 documentation
- Hugging Face blog: Gemma 4
- A Visual Guide to Gemma 4

문서 15

docs/15-troubleshooting-and-final-check.md

트러블슈팅 / 최종 체크 / 참고 링크

문제 발생 시 바로 해볼 것

Ollama

- `ollama --version`이 안 되면 앱 또는 설치 상태를 다시 확인
- 모델 로드 실패 시 더 작은 모델로 변경
- `gemma4:e4b` -> `gemma4:e2b`
- 메모리 부족 시 브라우저 탭과 다른 앱 종료
- 너무 느리면 큰 모델 대신 E2B/E4B 사용

LM Studio

- 앱이 켜지지만 모델이 안 돌면 런타임 상태 확인
- `Cmd/Ctrl + Shift + R`
- 모델 로드 실패 시 더 작은 모델 선택
- 8GB 환경이면 `google/gemma-4-e2b`만 사용
- 8GB Mac/노트북에서 시스템이 멈춘 것처럼 보이면, 잠시 기다린 뒤 앱을 종료하고 브라우저 탭과 다른 앱을 먼저 줄인 다음 다시 시도
- Intel Mac이면 LM Studio 대신 Ollama 사용

llama.cpp

- `llama-cli --version` 또는 `llama-server --version`이 안 되면 PATH 또는 설치 상태를 다시 확인
- 모델 경로 오류가 나면 `.gguf` 절대 경로를 다시 확인
- 모델이 너무 느리거나 메모리 부족이면 더 작은 모델이나 더 작은 quant 사용
- macOS / Linux 서버 확인은 `curl http://127.0.0.1:8080/v1/models`로 먼저 점검
- Windows PowerShell 서버 확인은 `Invoke-RestMethod http://127.0.0.1:8080/v1/models`가 더 안전함

Hermes Agent

- `hermes version`이 안 되면 셸을 다시 열거나 `source ~/.zshrc` / `source ~/.bashrc` 실행
- `hermes doctor`에서 누락 의존성이 보이면 먼저 그 결과를 기준으로 수정
- Windows 사용자는 네이티브가 아니라 WSL2 안에서 실행해야 함
- Windows에서 `hermes`가 안 잡히면 먼저 PowerShell에서 `wsl --status`, `wsl -l -v`를 확인하고, WSL2 Ubuntu 셸 안에서 다시 점검
- 로컬 모델 연결 실패 시 `hermes model`에서 base URL과 model ID를 다시 확인

ChromeOS

- Linux 개발 환경이 꺼져 있지 않은지 확인

- 학교/회사 정책으로 Linux가 막혀 있지 않은지 확인
- 로컬 LLM용 전용 GPU/VRAM 가속을 기대하기 어려우므로 성능 기대치를 낮추고 작은 모델만 사용

행사 전 최종 확인

아래 중 하나 이상이 되면 준비 완료입니다.

LM Studio 준비 완료 기준

- 앱 실행 성공
- `google/gemma-4-e2b` 또는 `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목 다운로드 완료
- Chat 화면에서 한 번 답변 생성 성공
- Windows에서 CLI까지 확인하고 싶다면 PowerShell에서 아래처럼 확인할 수 있습니다.

```powershell
lms --help
```

8GB 장비 주의사항

8GB 메모리 노트북은 작은 모델(E2B)에서도 속도가 많이 느리거나 시스템이 일시적으로 멈춘 것처럼 보일 수 있습니다.

- 가능하면 16GB 이상 메모리 장비를 권장합니다.
- 8GB 장비로 참석하는 경우에는 E2B만 미리 다운로드하세요.
- 다른 앱을 종료한 상태에서 행사 전에 반드시 1회 실행 테스트를 완료해 주세요.

Ollama 준비 완료 기준

macOS / Linux:

```bash
ollama --version
ollama run gemma4:e2b
```

Windows PowerShell:

```powershell
ollama --version
ollama run gemma4:e2b
```

llama.cpp 준비 완료 기준

macOS / Linux:

```bash
llama-cli --version
llama-server --version
```

Windows PowerShell:

```powershell
llama-cli --version
llama-server --version
```

그리고 아래 중 하나 이상:

- `llama-cli`로 1회 응답 생성 성공
- `llama-server` 실행 성공
- macOS / Linux: `curl http://127.0.0.1:8080/v1/models`
- Windows PowerShell: `Invoke-RestMethod http://127.0.0.1:8080/v1/models`

Hermes 준비 완료 기준

macOS / Linux / WSL2:

```bash
hermes version
hermes doctor
```

그 다음 `hermes` 또는 `hermes --tui`를 실행해 짧은 질문이 응답되는지 확인하세요.

로컬 모델을 붙여 볼 참가자는 `hermes model`에서 로컬 서버 주소 등록까지 확인하세요.

Windows 사용자는 위 명령을 PowerShell이 아니라 WSL2 Ubuntu 셸 안에서 실행해야 합니다.

공식 참고 링크

Gemma 4

- Gemma 4 출시 블로그 - Google Developers Blog

Ollama

- Ollama Quickstart
- Ollama macOS
- Ollama Windows
- Ollama Linux
- Ollama Gemma 4 Library
- Ollama Context Length

LM Studio

- LM Studio Download
- LM Studio Docs
- LM Studio System Requirements
- LM Studio Get Started
- LM Studio Download an LLM
- LM Studio CLI

- LM Studio Offline Operation
- LM Studio Gemma 4 Models
- LM Studio MLX Engine Blog

llama.cpp

- llama.cpp README
- llama.cpp Install Guide
- llama.cpp Build Guide

Hermes Agent

- Hermes Installation
- Hermes Quickstart
- Hermes CLI Commands Reference
- Hermes FAQ
- Hermes Docs Home

Apple MLX / Community MLX

- Apple MLX GitHub
- Apple MLX LM GitHub
- MLX LoRA / QLoRA Guide
- Hugging Face MLX Community

ChromeOS

- Set up Linux on your Chromebook